

Benemérita Universidad Autónoma de Puebla

Facultad de Ciencias de la Computación

Maquinas de Aprendizaje

Proyecto Final: Predicción del Clima



Docente: Abraham Sanchez Lopez

Alumno: Brandon Rodrigo Méndez Méndez

Matricula: 202130905

Primavera 2026

ÍNDICE

Introducción.....	4
Descripción del Dataset.....	4
Parte 1 - KDD y Modelos Bases.....	5
1.1 - KDD: Análisis Exploratorio de Datos (EDA).....	5
1.1.1 - Carga e Inspección Inicial.....	5
1.1.2 - Análisis de Valores Faltantes.....	9
1.1.3 - Hallazgos clave del análisis de valores faltantes.....	11
1.1.4 - Variable Objetivo: MeanTemp.....	12
1.1.5 - Análisis Temporal.....	13
1.1.5.1 - Temperatura media promedio por año.....	13
1.1.5.2 - Temperatura media promedio por mes.....	14
1.1.6 - Análisis de Correlaciones.....	14
1.1.7 - Análisis por Estación Meteorológica.....	17
1.1.8 - PCA exploratorio para selección de variables.....	19
1.1.8 - Skimr: Resumen Completo.....	22
1.2 - Preprocesamiento para Modelado.....	24
1.3 - División Train/Test y Normalización.....	25
1.4 - Análisis de Componentes Principales (PCA).....	26
1.4.1 - Varianza por componente.....	26
1.4.2 - Biplot de variables.....	27
1.4.3 - Gráfico de contribuciones a PC1 y PC2.....	28
1.5 - Función de Evaluación de Modelos.....	29
1.6 - Modelo Base 1: Regresión Lineal Múltiple.....	30
1.6.1 - Resultados del modelo LM original.....	30
1.6.2 - Análisis de residuales.....	32
1.6.4 - Comparación general.....	32
1.7 - Modelo Base 2: Support Vector Machine.....	33
1.7.1 Resultados del modelo SVR original.....	33
1.7.2 - Modelo SVR con PCA.....	33
1.7.3 - Comparación SVR original vs SVR con PCA.....	34
1.8 - Modelo Base 3: Regresión Penalizada — Ridge (Glmnet).....	34
1.8.1 - Ridge Regression original.....	35
1.8.2 - Ridge Regression con PCA.....	36
1.8.3 - Comparación Ridge original vs Ridge con PCA.....	36
1.9 - Modelo Base 4: KNN.....	37
1.9.1 - KNN Original.....	37
1.9.2 - KNN con PCA.....	39
1.9.3 - Comparación KNN original vs KNN con PCA.....	41
2.0 - Comparación de Modelos Base.....	41
Parte 2 - Mejora de Modelos.....	43
2.1 - Ingeniería de Características.....	43
2.2 - Análisis Post-FE y Correlación.....	44
2.3 - Split y Normalización Post FE.....	45

2.4 - PCA Post Feature Engineering.....	46
2.5 - Ajuste de Hiperparámetros — Modelo Mejorado 1: Ridge/Lasso.....	47
2.5.1 - Lasso con Feature Engineering.....	47
2.5.2 - Elastic Net con Feature Engineering.....	48
2.5.3 - Lasso con Feature Engineering + PCA.....	49
2.5.4 - Comparación de modelos mejorados.....	49
2.6 - Modelo Mejorado 2: SVR con Tuning.....	50
2.6.1 - SVR con Feature Engineering + Tuning.....	50
2.6.2 - SVR con Feature Engineering + PCA + Tuning.....	51
2.7 - Modelo Mejorado 3: Regresión Lineal con Interacciones.....	52
2.8 - Modelo Mejorado 4: KNN con Ingeniería de Características.....	53
2.8.1 - KNN con Feature Engineering.....	53
2.8.2 - KNN con Feature Engineering + PCA.....	56
2.8.3 - Comparación KNN Original, KNN PCA, KNN FE y KNN FE + PCA.....	58
2.9 - Modelos Avanzados.....	59
2.9.1 - Random Forest.....	59
2.9.2 - XGBoost.....	61
2.9.3 - Gradient Boosting (GBM).....	63
3.0 - Ensamblados.....	65
3.0.1 - Ensemble Random Forest + XGBoost.....	65
3.0.2 - Ensemble XGBoost + GBM.....	65
3.0.3 - Random Forest + XGBoost + GBM.....	66
3.1 - Comparación Final de Todos los Modelos.....	67
3.1.1 - RMSE.....	68
3.1.2 - Gráfica R^2	69
3.1.3 - Gráfica Real vs Predicho.....	70
3.2 - Conclusiones.....	71

Introducción

El clima es uno de los factores más influyentes en la vida diaria, impactando desde la agricultura y la aviación hasta la planeación de actividades cotidianas. La predicción precisa de variables meteorológicas representa un desafío importante dentro de la ciencia de datos, ya que el comportamiento del clima depende de múltiples factores que interactúan entre sí, como la precipitación, la presión atmosférica, la velocidad del viento, la temperatura mínima y máxima, entre otros.

Poder anticipar la temperatura media diaria permite comprender mejor las condiciones climáticas de una región, apoyar la toma de decisiones, optimizar recursos y mejorar la planeación de actividades en distintos sectores. A través de un proceso riguroso de limpieza, transformación y análisis exploratorio de datos, se buscará identificar patrones relevantes, analizar relaciones entre variables y seleccionar aquellas características que aporten mayor información para la predicción.

Descripción del Dataset

El dataset utilizado en este proyecto corresponde a un conjunto de registros meteorológicos históricos, los cuales contienen información diaria recopilada por diferentes estaciones meteorológicas. En total, el conjunto de datos está compuesto por 119,040 observaciones y 31 variables, lo que permite realizar un análisis amplio del comportamiento climático durante el período estudiado.

Los registros provienen de 159 estaciones meteorológicas, lo cual aporta diversidad geográfica a los datos y permite observar variaciones en las condiciones climáticas dependiendo de la ubicación de cada estación. El período temporal cubierto por el dataset comprende desde el 1 de enero de 1940 hasta el 9 de septiembre de 1945, por lo que se trata de información histórica organizada en formato de serie temporal.

Para este proyecto, el enfoque seleccionado es de regresión, ya que la variable objetivo es MeanTemp, que representa la temperatura media diaria en grados Celsius. Al tratarse de una variable numérica continua, el objetivo principal será construir modelos capaces de predecir el valor de la temperatura media diaria a partir de las demás variables meteorológicas disponibles en el dataset.

Características	Valor
Observaciones	119,040
Variables	31
Estaciones Meteorológicas	159
Periodo Temporal	1940 - 1945
Variable Objetivo	MeanTemp - temperatura media diaria
Tipo de Problema	Regresión

Parte 1 - KDD y Modelos Bases

1.1 - KDD: Análisis Exploratorio de Datos (EDA)

El proceso de KDD consiste en transformar datos brutos en información útil mediante una serie de etapas que permiten comprender, preparar y analizar el conjunto de datos antes de aplicar modelos de Machine Learning.

Durante esta etapa se realizó una inspección detallada del dataset meteorológico con el propósito de conocer su estructura, identificar el tipo de variables disponibles, analizar la calidad de los datos y detectar posibles valores faltantes, duplicados o inconsistencias. Además, se exploraron las distribuciones de las variables climáticas y sus relaciones con la variable objetivo, con el fin de reconocer patrones que pudieran influir en la temperatura media diaria.

Este análisis previo permite tomar decisiones adecuadas sobre la limpieza, transformación y selección de variables, asegurando que los datos utilizados en la etapa de modelado sean más confiables y representativos. De esta manera, el KDD funciona como una base esencial para el desarrollo de modelos predictivos más precisos y mejor fundamentados.

1.1.1 - Carga e Inspección Inicial

En esta etapa se realiza la carga del dataset y una primera revisión de su estructura general. Primero se importan las librerías necesarias para el análisis, limpieza, visualización y modelado de datos. Posteriormente, se carga el archivo Summary_of_Weather.csv en RStudio para comenzar el proceso de exploración.

Después de cargar los datos, se ejecutará una inspección inicial para conocer sus dimensiones, visualizar las primeras filas, identificar el tipo de datos de cada variable y obtener un resumen estadístico general. Esto permite verificar que el dataset se haya cargado correctamente y comenzar a detectar posibles valores faltantes, variables numéricas, categóricas o inconsistencias que deban tratarse antes del modelado.

```
> # Instalación y carga de librerías
if (!require("pacman")) install.packages("pacman")
pacman::p_load(
  tidyverse, skimr, DataExplorer, corrplot, GGally,
  ggplot2, gridExtra, naniar, lubridate, scales,
  caret, glmnet, e1071, pROC, factoextra, FactoMineR,
  ROSE, DMwR2, randomForest, xgboost, gbm, Metrics
)
```

```
> # Carga del dataset
> df_raw <- read_csv("C:/Users/Brandon Méndez/Documents/Semestre 10/Maquinas de
Aprendizaje/ProyectoFinal/Summary of Weather.csv", show_col_types = FALSE)
```

Ahora se realizó una primera exploración del dataset para conocer su estructura general antes de iniciar la limpieza y el análisis exploratorio. Para ello, se utilizaron funciones básicas de R que permiten revisar el tamaño del conjunto de datos, visualizar sus primeros registros, identificar los tipos de variables y obtener un resumen estadístico inicial.

Primero, con `dim(df_raw)` se identificó que el dataset contiene 119,040 observaciones y 31 variables. Esto confirma que se trabaja con un conjunto de datos amplio, adecuado para aplicar técnicas de análisis y modelos de Machine Learning.

```
> # Inspección inicial
> dim(df_raw)
[1] 119040    31
>
```

Después, mediante `head(df_raw, 5)` se visualizaron las primeras cinco filas del dataset. Esta revisión permitió observar algunas variables importantes como STA, Date, Precip, WindGustSpd, MaxTemp, MinTemp, MeanTemp y Snowfall, así como detectar desde el inicio la presencia de valores faltantes en algunas columnas.

```
> head(df_raw, 5)
# A tibble: 5 x 31
  STA Date Precip WindGustSpd MaxTemp MinTemp MeanTemp Snowfall
  <dbl> <chr> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 10001 1942-7-1 1.016 NA 25.6 22.2 23.9 0
2 10001 1942-7-2 0 NA 28.9 21.7 25.6 0
3 10001 1942-7-3 2.54 NA 26.1 22.2 24.4 0
4 10001 1942-7-4 2.54 NA 26.7 22.2 24.4 0
5 10001 1942-7-5 0 NA 26.7 21.7 24.4 0
# i 23 more variables: PoorWeather <chr>, YR <dbl>, MO <dbl>, DA <dbl>,
# PRCP <chr>, DR <dbl>, SPD <dbl>, MAX <dbl>, MIN <dbl>, MEA <dbl>,
# SNF <dbl>, SND <dbl>, FT <lgl>, FB <lgl>, FTI <lgl>, ITH <lgl>,
# PGT <dbl>, TSHDSBRSGF <chr>, SD3 <lgl>, RHX <lgl>, RHN <lgl>, RVG <lgl>,
# WTE <lgl>
```

Posteriormente, con `glimpse(df_raw)` se revisó el tipo de dato de cada variable. Se observó que algunas columnas numéricas fueron leídas correctamente, mientras que otras, como Precip, aparecen como texto debido a posibles valores especiales o inconsistencias en el formato. Esto indica que será necesario realizar transformaciones durante la etapa de limpieza.

```

> glimpse(df_raw)
Rows: 119,040
Columns: 31
$ STA      <dbl> 10001, 10001, 10001, 10001, 10001, 10001, 10001, 10001,...
$ Date     <chr> "1942-7-1", "1942-7-2", "1942-7-3", "1942-7-4", "1942-7-5", ...
$ Precip   <chr> "1.016", "0", "2.54", "2.54", "0", "0", "T", "3.556", "0", ...
$ WindGustSpd <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ MaxTemp  <dbl> 25.55556, 28.88889, 26.11111, 26.66667, 26.66667, 26.66667, ...
$ MinTemp  <dbl> 22.22222, 21.66667, 22.22222, 22.22222, 21.66667, 21.66667, ...
$ MeanTemp <dbl> 23.88889, 25.55556, 24.44444, 24.44444, 24.44444, 24.44444, ...
$ Snowfall <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
$ PoorWeather <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ YR       <dbl> 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 42, ...
$ MO       <dbl> 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 7, ...
$ DA       <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, ...
$ PRCP     <chr> "0.04", "0", "0.1", "0.1", "0", "0", "T", "0.14", "T", "0", ...
$ DR       <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ SPD      <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ MAX      <dbl> 78, 84, 79, 80, 80, 80, 83, 80, 81, 78, 78, 76, 79, 79, 79, 79, ...
$ MIN      <dbl> 72, 71, 72, 72, 71, 71, 73, 72, 73, 71, 71, 72, 71, 72, 72, 72, ...
$ MEA      <dbl> 75, 78, 76, 76, 76, 76, 78, 76, 77, 74, 74, 74, 75, 76, 76, 76, ...
$ SNF      <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
$ SND      <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ FT       <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ FB       <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ FTI      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ ITH      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ PGT      <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ TSHDSBRSGF <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ SD3      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ RHX      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ RHN      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ RVG      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ WTE      <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
>

```

Finalmente, con `summary(df_raw)` se obtuvo un resumen estadístico general de las variables. En este resumen se identificaron rangos mínimos y máximos, medias, medianas y cantidad de valores faltantes. Destaca que la variable objetivo `MeanTemp` presenta valores numéricos continuos, lo que confirma que el problema será abordado como una tarea de regresión.

```
> summary(df_raw)
```

STA	Date	Precip	WindGustSpd
Min. :10001	Length:119040	Length:119040	Min. :18.52
1st Qu.:11801	Class :character	Class :character	1st Qu.:29.63
Median :22508	Mode :character	Mode :character	Median :37.04
Mean :29659			Mean :37.77
3rd Qu.:33501			3rd Qu.:43.06
Max. :82506			Max. :75.93
			NA's :118508
MaxTemp	MinTemp	MeanTemp	Snowfall
Min. : -33.33	Min. : -38.33	Min. : -35.56	Min. : 0.0000
1st Qu.: 25.56	1st Qu.: 15.00	1st Qu.: 20.56	1st Qu.: 0.0000
Median : 29.44	Median : 21.11	Median : 25.56	Median : 0.0000
Mean : 27.05	Mean : 17.79	Mean : 22.41	Mean : 0.2455
3rd Qu.: 31.67	3rd Qu.: 23.33	3rd Qu.: 27.22	3rd Qu.: 0.0000
Max. : 50.00	Max. : 34.44	Max. : 40.00	Max. :86.3600
			NA's :1207
PoorWeather	YR	MO	DA
Length:119040	Min. :40.00	Min. : 1.000	Min. : 1.0
Class :character	1st Qu.:43.00	1st Qu.: 4.000	1st Qu.: 8.0
Mode :character	Median :44.00	Median : 7.000	Median :16.0
	Mean :43.81	Mean : 6.726	Mean :15.8
	3rd Qu.:45.00	3rd Qu.:10.000	3rd Qu.:23.0
	Max. :45.00	Max. :12.000	Max. :31.0
PRCP	DR	SPD	MAX
Length:119040	Min. : 2	Min. :10.00	Min. : -28
Class :character	1st Qu.:11	1st Qu.:16.00	1st Qu.: 78
Mode :character	Median :32	Median :20.00	Median : 85
	Mean :27	Mean :20.40	Mean : 81
	3rd Qu.:34	3rd Qu.:23.25	3rd Qu.: 89
	Max. :78	Max. :41.00	Max. :122
	NA's :118507	NA's :118508	NA's :474
MIN	MEA	SNF	SND
Min. : -37.00	Min. : -32.00	Min. :0.00000	Min. :0
1st Qu.: 59.00	1st Qu.: 69.00	1st Qu.:0.00000	1st Qu.:0
Median : 70.00	Median : 78.00	Median :0.00000	Median :0
Mean : 64.27	Mean : 72.64	Mean :0.00967	Mean :0
3rd Qu.: 74.00	3rd Qu.: 81.00	3rd Qu.:0.00000	3rd Qu.:0
Max. : 94.00	Max. :104.00	Max. :3.40000	Max. :0
NA's :468	NA's :498	NA's :1207	NA's :113477
FT	FB	FTI	ITH
Mode:logical	Mode:logical	Mode:logical	Mode:logical
NA's:119040	NA's:119040	NA's:119040	NA's:119040

PGT	TSHDSBRS GF	SD3	RHX
Min. : 0.00	Length:119040	Mode:logical	Mode:logical
1st Qu.: 8.50	Class :character	NA's:119040	NA's:119040
Median :11.60	Mode :character		
Mean :12.09			
3rd Qu.:15.00			
Max. :23.90			
NA's :118515			
RHN	RVG	WTE	
Mode:logical	Mode:logical	Mode:logical	
NA's:119040	NA's:119040	NA's:119040	

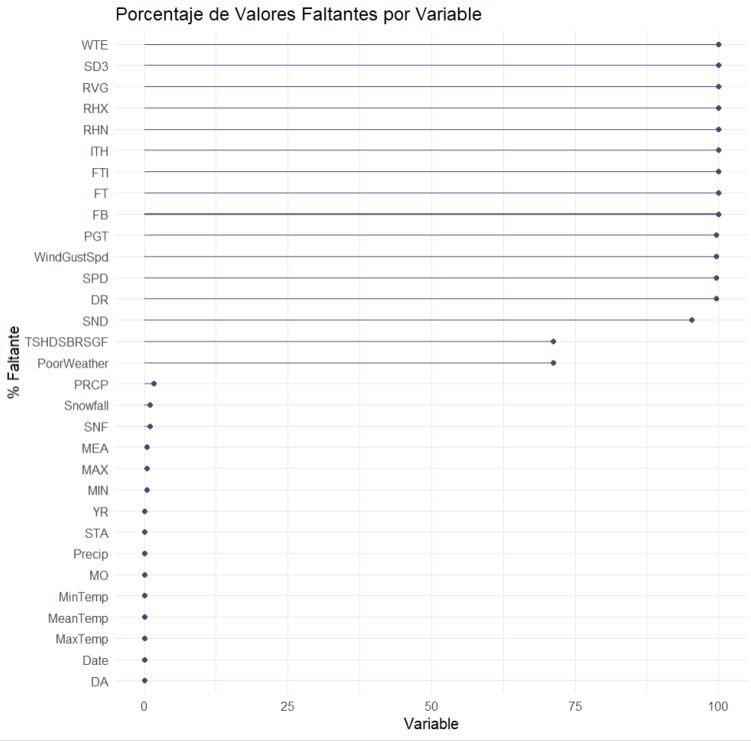
1.1.2 - Análisis de Valores Faltantes

En esta etapa se revisó el porcentaje de valores faltantes por cada variable del dataset. Para ello, se calculó el porcentaje de datos nulos en todas las columnas y se ordenaron de mayor a menor, con el objetivo de identificar qué variables presentan más pérdida de información.

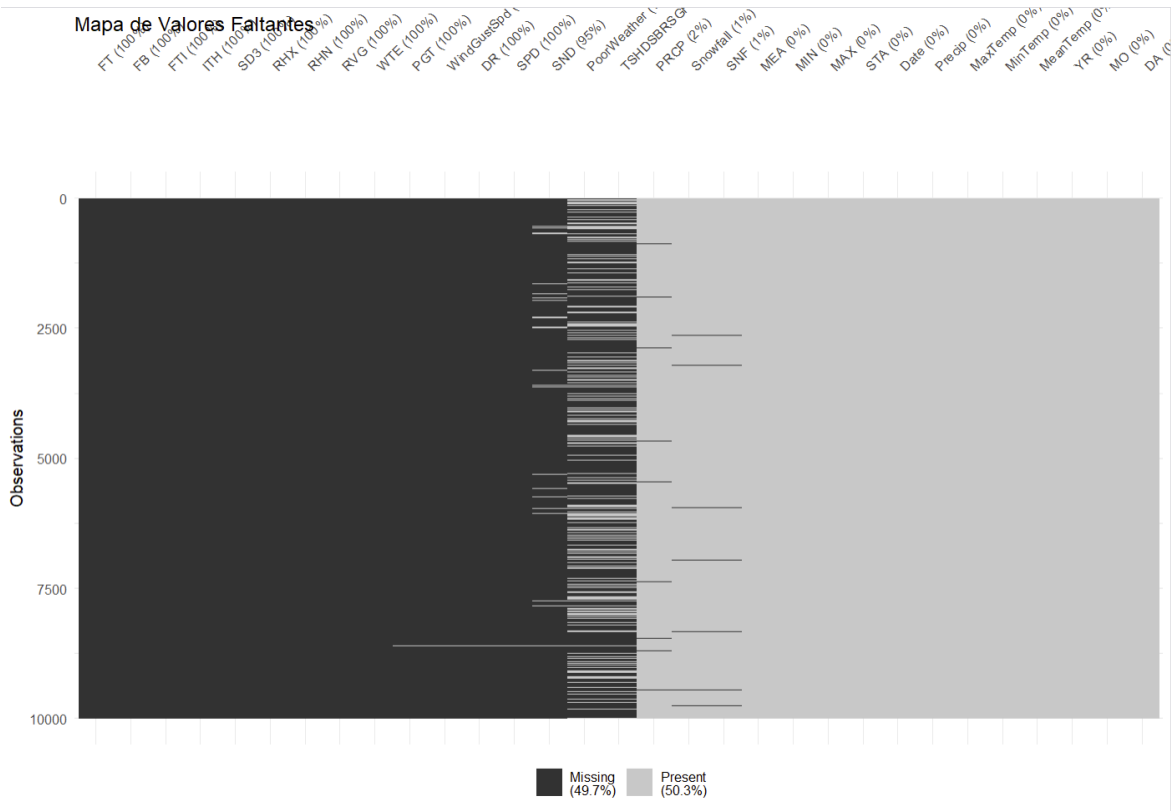
Los resultados muestran que varias variables como FT, FB, FTI, ITH, SD3, RHX, RHN, RVG y WTE tienen 100% de valores faltantes, por lo que no aportan información útil para el análisis. También se identificaron variables con un porcentaje muy alto de nulos, como PGT, WindGustSpd, SPD, DR y SND, las cuales superan el 95% de datos faltantes.

```
> print(null_pct, n = 31)
# A tibble: 31 x 2
  Variable      Pct_Nulos
  <chr>         <dbl>
1 FT           100
2 FB           100
3 FTI          100
4 ITH          100
5 SD3          100
6 RHX          100
7 RHN          100
8 RVG          100
9 WTE          100
10 PGT          99.6
11 WindGustSpd  99.6
12 SPD          99.6
13 DR           99.6
14 SND          95.3
15 PoorWeather  71.2
16 TSHDSBRS GF  71.2
17 PRCP         1.62
18 Snowfall     1.01
19 SNF          1.01
20 MEA          0.418
21 MAX          0.398
22 MIN          0.393
23 STA          0
24 Date         0
25 Precip       0
26 MaxTemp      0
27 MinTemp      0
28 MeanTemp     0
29 YR           0
30 MO           0
31 DA           0
```

Por otro lado, variables importantes para el modelo de regresión, como MeanTemp, MaxTemp, MinTemp, Date, STA, YR, MO y DA, no presentan valores faltantes, lo cual es favorable porque permite conservar información relevante para predecir la temperatura media diaria.



Las gráficas utilizadas permiten visualizar de forma más clara la distribución de los valores faltantes. El gráfico de porcentaje muestra qué variables tienen mayor ausencia de datos, mientras que el mapa de valores faltantes permite observar el patrón general de datos disponibles y ausentes dentro del dataset.



1.1.3 - Hallazgos clave del análisis de valores faltantes

A partir del cálculo del porcentaje de valores nulos y de las visualizaciones realizadas, se identificaron los siguientes hallazgos:

- **9 variables presentan 100% de valores faltantes:**
FT, FB, FTI, ITH, SD3, RHX, RHN, RVG y WTE. Estas variables no aportan información útil al análisis, por lo que se recomienda eliminarlas del dataset.
- **PGT, WindGustSpd, SPD y DR presentan aproximadamente 99.6% de valores faltantes.**
Debido a que casi toda su información está ausente, estas variables también se consideran poco confiables para el modelado y deben eliminarse.
- **SND presenta 95.3% de valores faltantes.**
Aunque no está completamente vacía, la cantidad de datos disponibles es muy baja, por lo que conservarla podría introducir ruido o sesgos en el modelo.
- **PoorWeather y TSHDSBRS GF tienen 71.2% de valores faltantes.**
Estas variables presentan una pérdida importante de información. Además, como el enfoque actual del proyecto es de **regresión** y no de clasificación, la variable **PoorWeather** pierde relevancia como posible variable objetivo.
- **Las variables principales para el modelo no presentan valores faltantes.**
Variables como **MeanTemp, MaxTemp, MinTemp, Precip, STA, Date, YR, MO y DA** tienen **0% de nulos**, lo cual es favorable para el desarrollo del modelo de regresión.
- **La variable objetivo MeanTemp no contienen valores faltantes.**
Esto es importante porque permite utilizar todos los registros disponibles para entrenar y evaluar los modelos sin necesidad de eliminar observaciones por ausencia en la variable objetivo.
- **El patrón de valores faltantes no es uniforme.**
El mapa de valores faltantes muestra que la ausencia de datos se concentra principalmente en ciertas columnas, no de forma aleatoria en todo el dataset. Esto justifica la eliminación de variables con alta pérdida de información en lugar de eliminar filas completas.
- **La limpieza del dataset**
Se debe enfocarse en eliminar columnas con alto porcentaje de nulos y conservar las variables meteorológicas completas o casi completas. Esto permitirá trabajar con un conjunto de datos más confiable, reduciendo ruido y mejorando la calidad de entrada para los modelos de regresión.

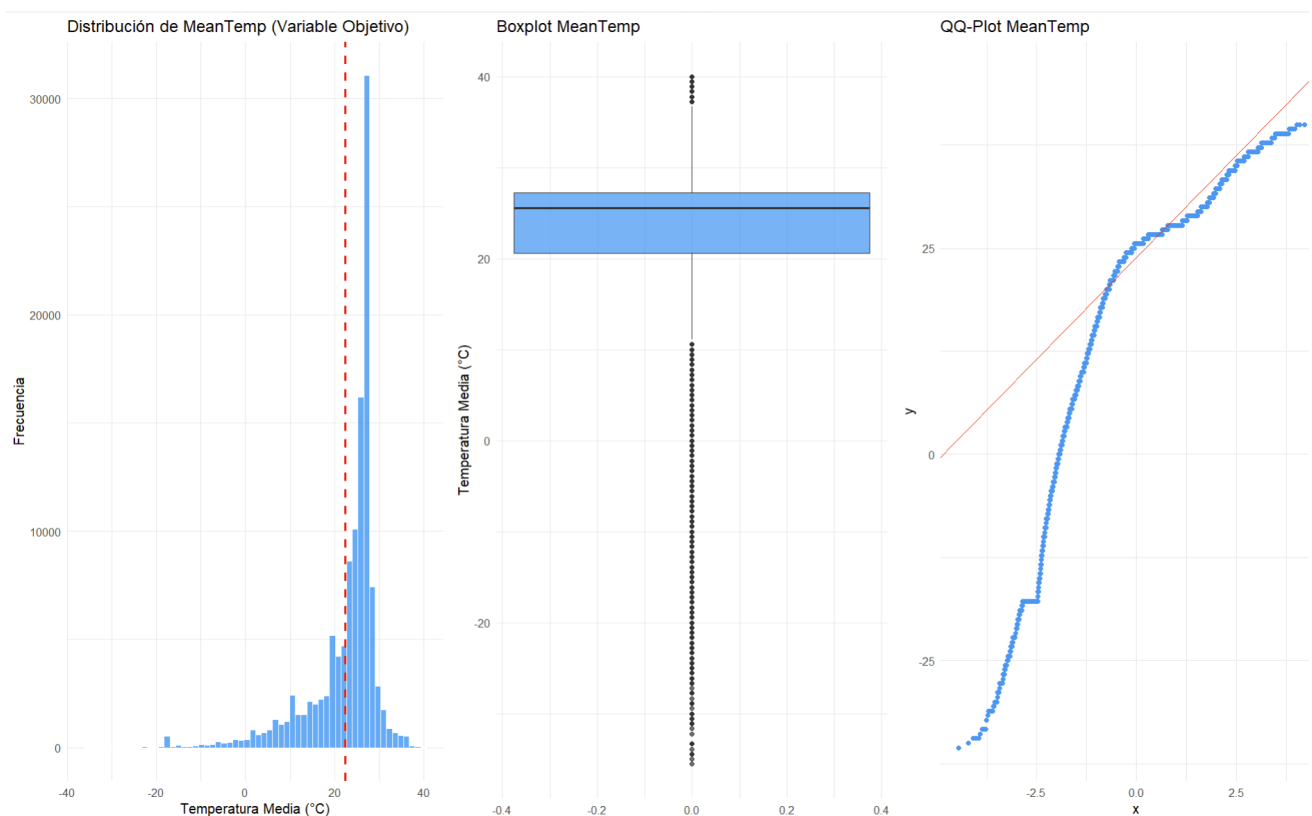
1.1.4 - Variable Objetivo: MeanTemp

En esta etapa se analizó la distribución de la variable objetivo MeanTemp, que representa la temperatura media diaria en grados Celsius. Para ello, se utilizaron tres visualizaciones, un histograma, un boxplot y un QQ-Plot.

Además de las gráficas, se calcularon estadísticos descriptivos de la variable MeanTemp para comprender mejor su comportamiento. La temperatura media diaria presenta una media de 22.41 °C y una mediana de 25.56 °C. El hecho de que la mediana sea mayor que la media indica que existen valores bajos que reducen el promedio general.

La desviación estándar es de 8.30 °C, lo que muestra que existe una variabilidad considerable en las temperaturas registradas. Esto es coherente con el boxplot, donde se observan valores atípicos, especialmente en temperaturas bajas.

Estos resultados confirman que la variable objetivo no tiene una distribución completamente uniforme ni perfectamente normal. Por ello, es importante considerar la presencia de valores extremos durante la etapa de limpieza y modelado, ya que pueden influir en el desempeño de los modelos de regresión.



1.1.5 - Análisis Temporal

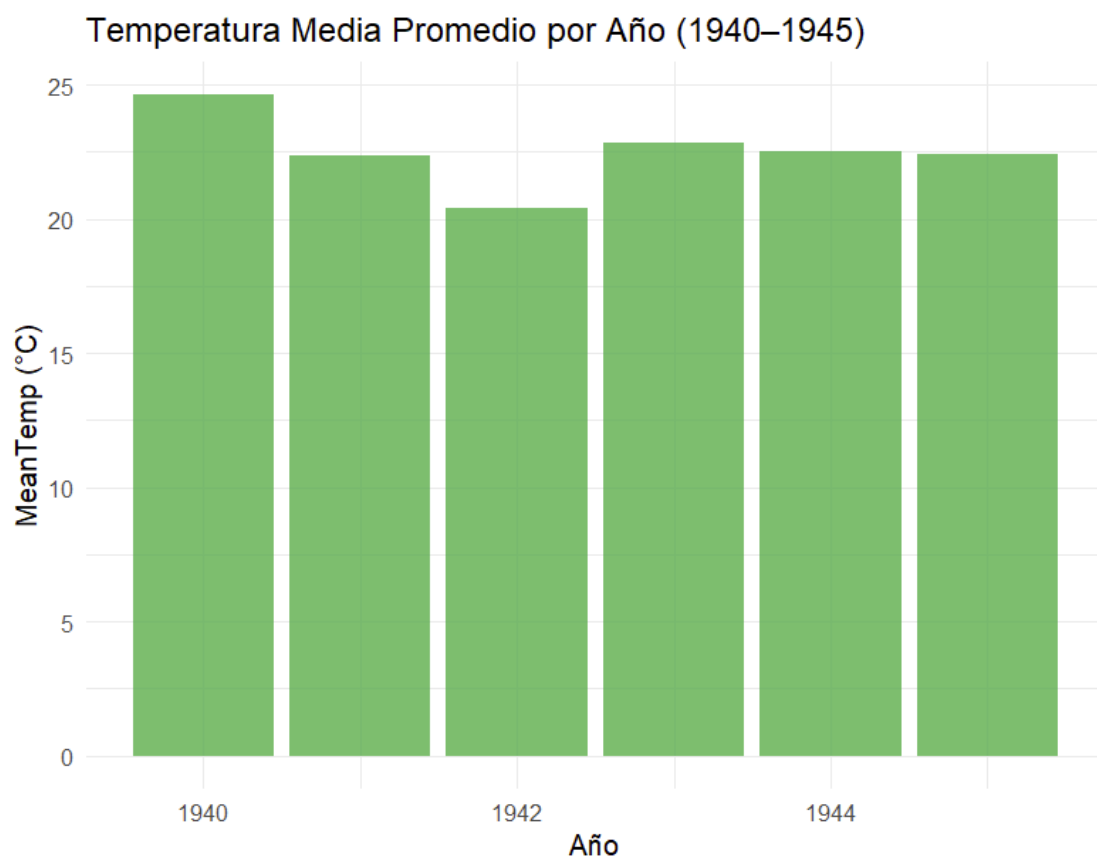
En esta etapa se transformó la variable Date a formato de fecha para poder extraer el mes y el año de cada registro. Esto permitió analizar cómo cambia la variable objetivo MeanTemp a lo largo del tiempo.

1.1.5.1 - Temperatura media promedio por año

La primera gráfica muestra la temperatura media promedio por año durante el período de 1940 a 1945. Se observa que las temperaturas promedio anuales se mantienen relativamente estables, en un rango aproximado entre 20 °C y 25 °C.

El año 1940 presenta el promedio más alto, cercano a los 25 °C, mientras que 1942 muestra el promedio más bajo, alrededor de 20.5 °C. Después de 1942, la temperatura promedio vuelve a incrementarse y se mantiene estable entre 1943 y 1945.

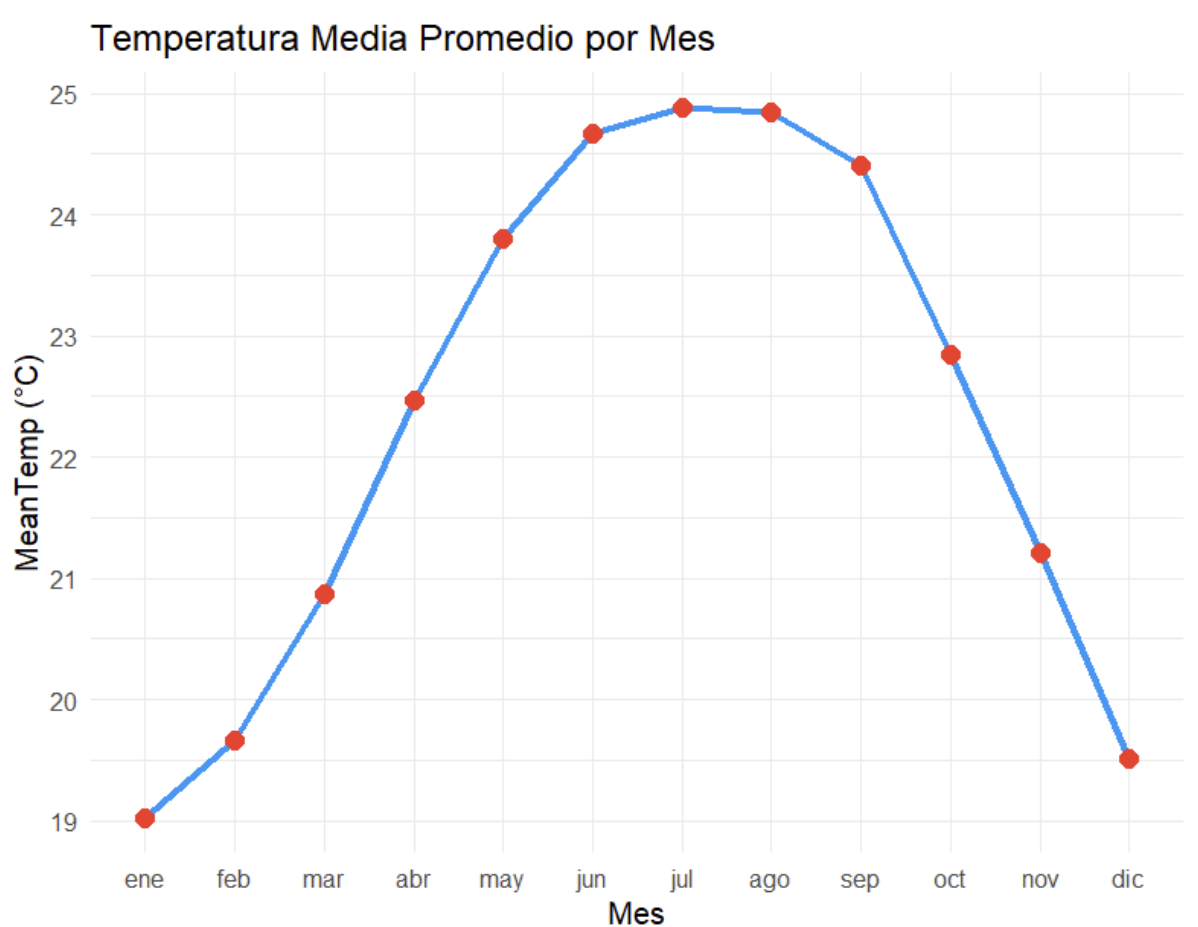
Esto permite identificar posibles variaciones anuales dentro del dataset, aunque en general no se observa una tendencia creciente o decreciente muy marcada.



1.1.5.2 - Temperatura media promedio por mes

La segunda gráfica muestra el comportamiento promedio de MeanTemp por mes. Aquí se observa un patrón estacional claro: las temperaturas son más bajas al inicio y al final del año, especialmente en enero, febrero y diciembre, mientras que aumentan progresivamente durante la primavera y alcanzan sus valores más altos entre junio, julio y agosto.

Este comportamiento indica que el mes puede ser una variable importante para el modelo de regresión, ya que existe una relación evidente entre la época del año y la temperatura media diaria.



1.1.6 - Análisis de Correlaciones

Para identificar qué variables tienen mayor relación con la variable objetivo MeanTemp, se seleccionaron las variables numéricas más relevantes y con menor porcentaje de valores faltantes. Posteriormente, se aplicó un análisis de correlación con el propósito de conocer la fuerza de relación lineal entre las variables del dataset.

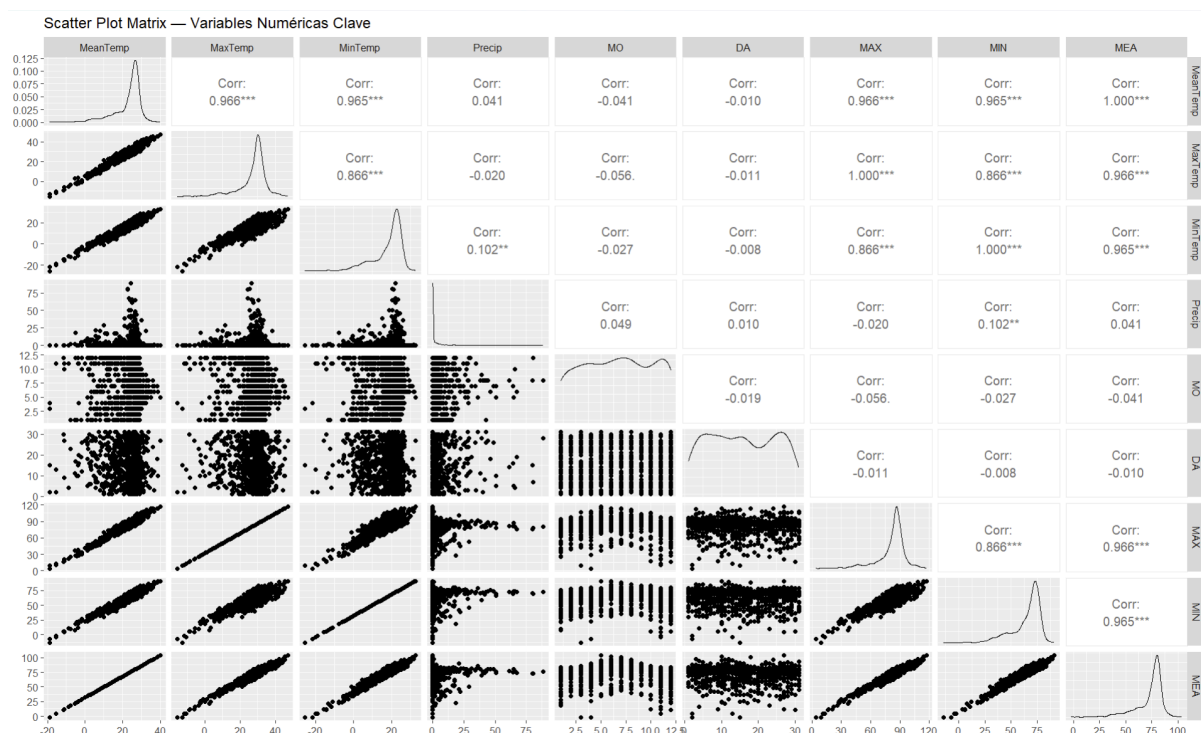
Este análisis es importante porque permite detectar qué variables pueden aportar más información al modelo de regresión, así como identificar posibles variables redundantes o repetidas que podrían afectar el entrenamiento del modelo.

También se observa que las variables MAX, MIN y MEA presentan correlaciones muy altas con las variables de temperatura en grados Celsius. Esto se debe a que representan mediciones equivalentes, pero en otra escala de temperatura. En especial, MEA tiene una correlación de 1.00 con MeanTemp, por lo que no debe utilizarse como variable predictora, ya que contiene prácticamente la misma información que la variable objetivo y podría generar fuga de información.

[illegible]

En esta visualización se confirma que MeanTemp, MaxTemp y MinTemp mantienen una relación lineal fuerte, ya que los puntos forman una tendencia ascendente clara. Esto significa que, cuando aumentan la temperatura máxima o mínima, también tiende a aumentar la temperatura media.

En cambio, Precip, MO y DA muestran una dispersión más amplia y sin una tendencia lineal clara respecto a MeanTemp. Esto indica que su relación directa con la temperatura media es débil, aunque algunas de ellas pueden conservarse si aportan información temporal o contextual al modelo.



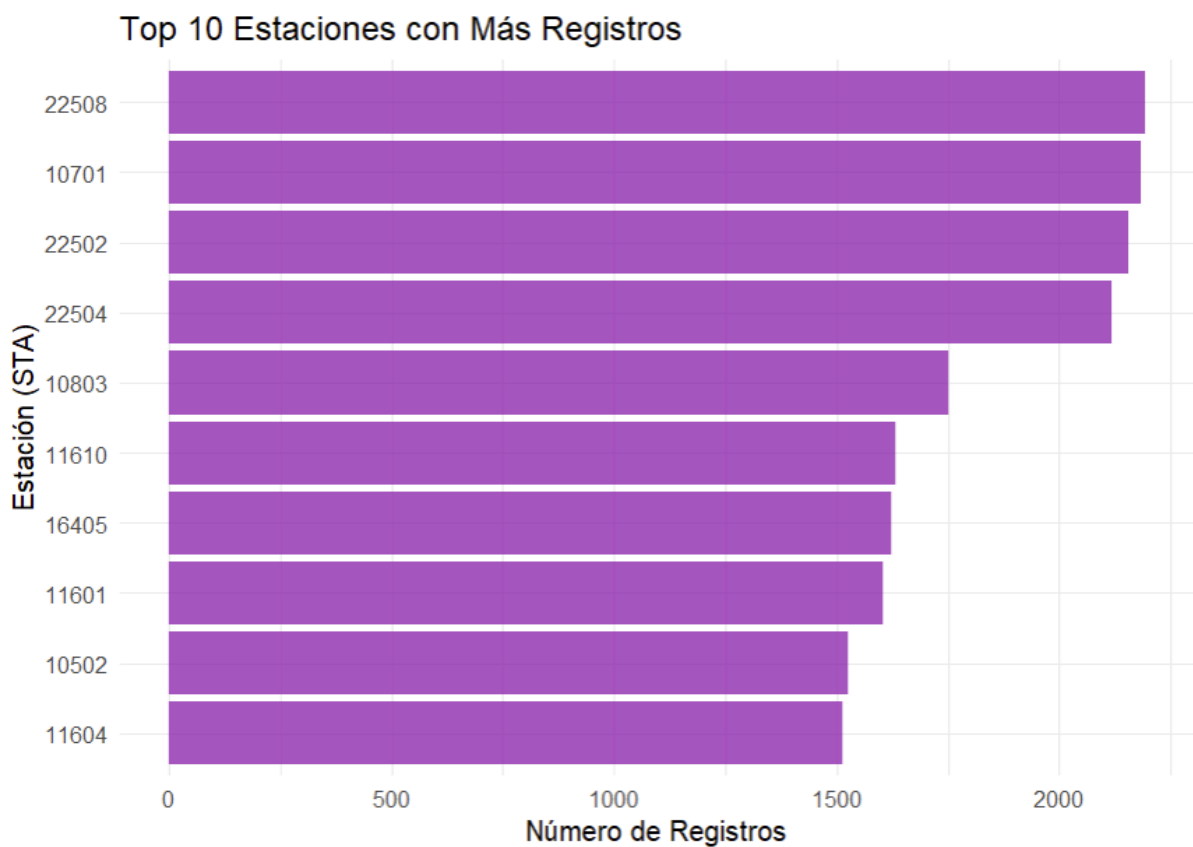
1.1.7 - Análisis por Estación Meteorológica

Este análisis permite observar cómo se distribuyen los registros entre las diferentes estaciones meteorológicas y cómo varía la temperatura media dependiendo de cada estación. Esto es importante porque el dataset contiene información de múltiples estaciones, por lo que no todas aportan la misma cantidad de datos ni presentan el mismo comportamiento climático.

La primera gráfica muestra las 10 estaciones meteorológicas con mayor cantidad de registros dentro del dataset. Se observa que algunas estaciones, como 22508, 10701, 22502 y 22504, concentran un número alto de observaciones, superando aproximadamente los 2,000 registros.

Esto indica que la distribución de datos no es completamente uniforme entre estaciones, ya que algunas tienen más registros disponibles que otras. Este punto es importante porque las estaciones con mayor cantidad de datos pueden tener más influencia durante el entrenamiento del modelo, especialmente si no se controla el balance de registros por estación.

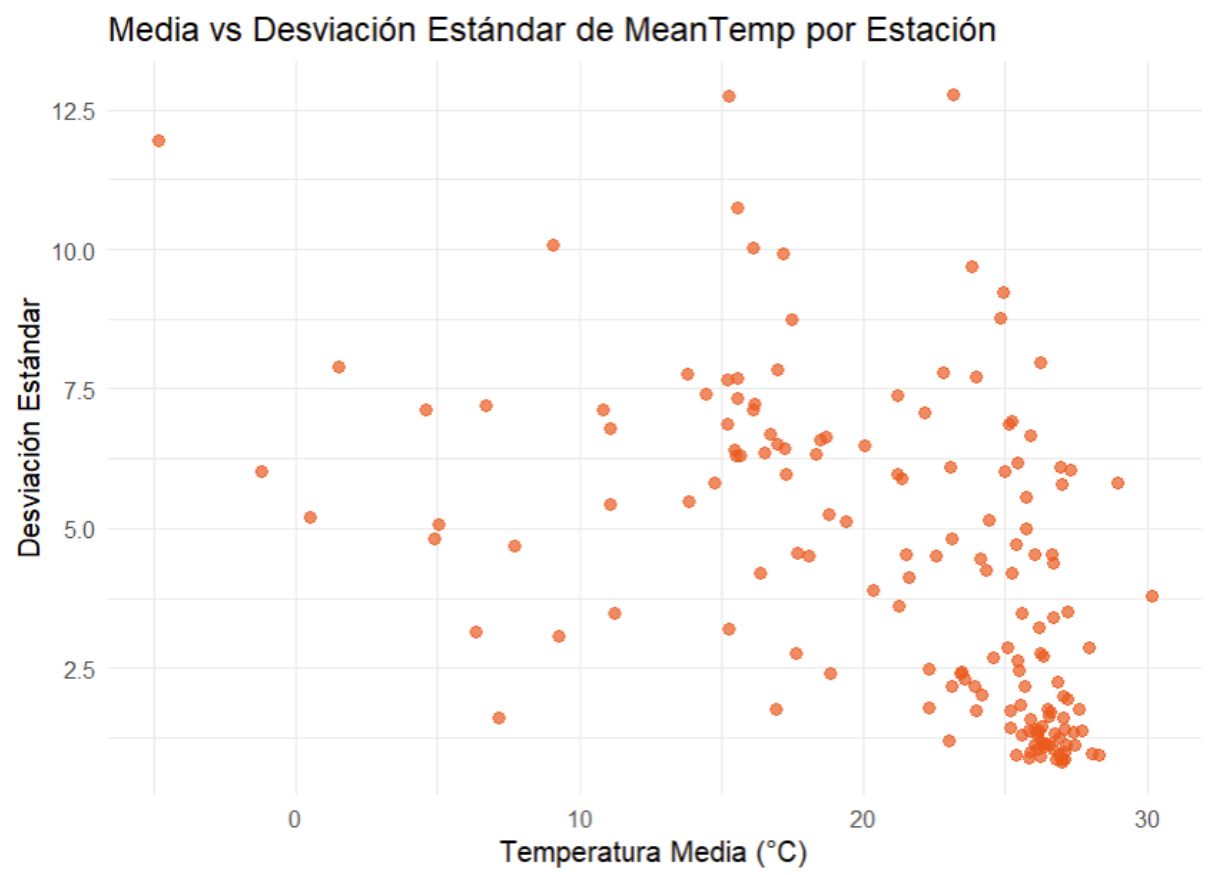
En el contexto del modelo de regresión, la variable STA puede aportar información relevante, ya que cada estación puede representar una ubicación geográfica distinta con condiciones climáticas particulares.



La segunda gráfica muestra la relación entre la temperatura media promedio y la desviación estándar de MeanTemp para cada estación meteorológica. Cada punto representa una estación, permitiendo observar tanto el nivel promedio de temperatura como la variabilidad que presenta a lo largo del tiempo.

Se observa que muchas estaciones se concentran entre aproximadamente 20 °C y 28 °C, con desviaciones estándar relativamente bajas. Esto indica que varias estaciones presentan temperaturas medias estables. Sin embargo, también existen estaciones con mayor variabilidad, lo que sugiere cambios más marcados en la temperatura registrada.

Esta información es útil porque permite identificar que no todas las estaciones tienen el mismo comportamiento térmico. Algunas presentan temperaturas más constantes, mientras que otras muestran mayor dispersión, posiblemente por diferencias de ubicación, altitud, temporada o cantidad de registros.



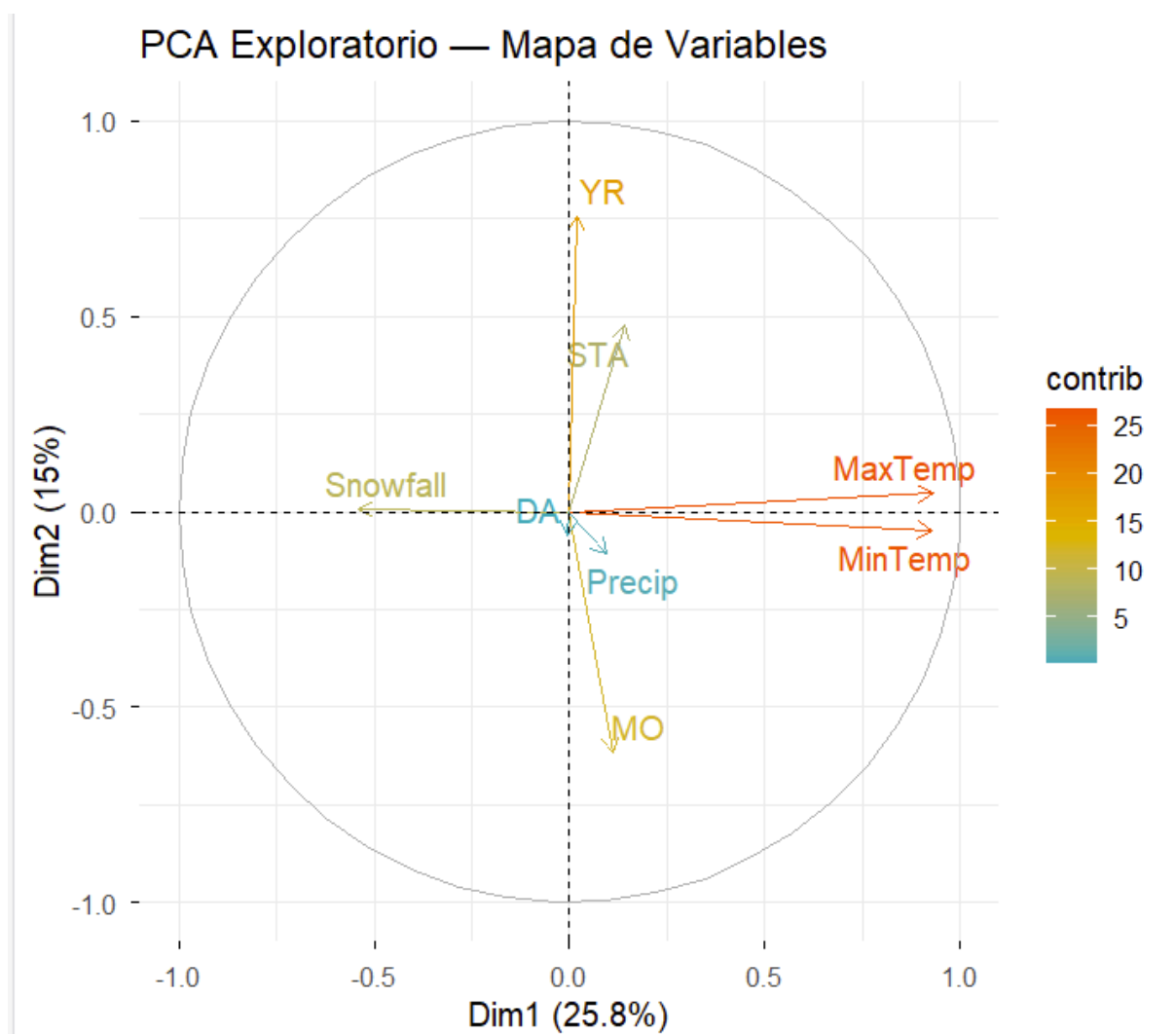
Estas gráficas muestran que existe variación en la cantidad de registros y en el comportamiento de la temperatura entre estaciones meteorológicas. Por ello, la variable STA puede ser relevante para el modelo de regresión, ya que ayuda a representar diferencias entre ubicaciones. Además, este análisis permite detectar estaciones con mayor influencia en el dataset y estaciones con mayor variabilidad térmica.

1.1.8 - PCA exploratorio para selección de variables

Se realizó un PCA exploratorio con el objetivo de analizar la relevancia de las variables predictoras y conocer cuáles aportan mayor información a la estructura general del dataset. Esta técnica permite transformar las variables originales en componentes principales, los cuales resumen la mayor cantidad posible de variabilidad de los datos.

En este caso, el PCA se utilizó como una herramienta de apoyo para identificar variables importantes y posibles variables candidatas a eliminación. Antes de aplicarlo, las variables fueron normalizadas, ya que el PCA es sensible a la escala de los datos. Esto significa que, si una variable tiene valores mucho más grandes que otra, podría dominar el análisis de forma incorrecta.

Es importante mencionar que el PCA no selecciona variables directamente en función de la variable objetivo MeanTemp, sino que muestra qué variables explican mayor variabilidad dentro del conjunto de datos. Por esta razón, sus resultados se utilizaron como complemento junto con otros criterios, como valores faltantes, correlación, redundancia y desempeño de los modelos.



La gráfica del mapa de variables muestra la relación de cada variable con los dos primeros componentes principales. El eje horizontal representa Dim1, que explica aproximadamente el 25.8% de la variabilidad, mientras que el eje vertical representa Dim2, que explica aproximadamente el 15%.

En la gráfica se observa que MaxTemp y MinTemp tienen flechas largas y apuntan en una dirección muy similar sobre el eje de Dim1. Esto indica que ambas variables aportan fuertemente al primer componente principal y están altamente relacionadas entre sí. Por lo tanto, se confirma que las variables de temperatura son de las más relevantes dentro del dataset.

Por otro lado, YR, MO y STA se orientan principalmente hacia Dim2, lo que indica que estas variables aportan información relacionada con el tiempo y la estación meteorológica. Esto es importante porque permite identificar que no solo las variables térmicas explican la estructura de los datos, sino también variables temporales y de ubicación.

La variable Snowfall aparece en una dirección distinta a las variables de temperatura, lo que sugiere que aporta información diferente. En cambio, Precip y DA aparecen más cerca del centro o con menor longitud, lo que indica que tienen menor contribución a los primeros componentes principales.

Variable	Contribución Dim 1	Contribución Dim 2	Acomulada PC1 + PC2
YR	0.0151	47.5499	47.5651
MaxTemp	41.9700	0.1819	42.1529
MinTemp	41.8876	0.1919	42.0795
MO	0.5865	31.7746	32.3611
STA	0.9929	19.0416	20.0346
Snowfall	14.1081	0.0028	14.1109
Precip	0.4361	0.9352	1.3713
DA	0.0026	0.3220	0.3245

La tabla de contribuciones muestra de forma numérica cuánto aporta cada variable a Dim1 y Dim2. Además, se calculó una contribución acumulada llamada Contrib_PC1_PC2, que suma el aporte de cada variable en los dos primeros componentes principales.

Los resultados muestran que las variables con mayor contribución acumulada fueron:

- YR, con aproximadamente 47.56%.
- MaxTemp, con aproximadamente 42.15%.
- MinTemp, con aproximadamente 42.08%.
- MO, con aproximadamente 32.36%.
- STA, con aproximadamente 20.03%.
- Snowfall, con aproximadamente 14.11%.

Esto indica que estas variables son las que más aportan a la estructura principal de los datos según el PCA. En particular, MaxTemp y MinTemp destacan por su alta contribución al primer componente, mientras que YR, MO y STA aportan principalmente al segundo componente.

Por otro lado, Precip y DA presentaron contribuciones bajas, con valores aproximados de 1.37% y 0.32%, respectivamente. Esto sugiere que, desde el punto de vista del PCA, estas variables tienen menor relevancia en los dos primeros componentes principales y podrían considerarse candidatas a eliminación o variables de baja prioridad.

Sin embargo, no se recomienda eliminarlas únicamente con base en el PCA, ya que algunas variables con baja contribución pueden aportar información útil en modelos no lineales como Random Forest o XGBoost. Por ello, el PCA se utilizó como criterio de apoyo y no como único método de selección.

En conclusión, el PCA exploratorio permitió identificar que las variables con mayor aportación fueron YR, MaxTemp, MinTemp, MO, STA y Snowfall. Estas variables explican una parte importante de la variabilidad del dataset y, por lo tanto, son relevantes para conservarse en el proceso de modelado.

En cambio, Precip y DA mostraron una baja contribución en los primeros componentes principales, por lo que pueden considerarse variables de menor prioridad. Aun así, su eliminación debe evaluarse con cuidado, considerando otros análisis y el desempeño de los modelos predictivos.

1.1.8 - Skimr: Resumen Completo

En esta etapa se utilizó la función `skim()` para obtener un resumen más completo del dataset. Esta herramienta permite revisar de forma organizada la cantidad de filas y columnas, los tipos de variables, el porcentaje de datos completos, los valores faltantes y algunos estadísticos descriptivos.

El resumen confirma que el dataset contiene 119,040 registros. Después de crear las variables temporales `Mes` y `Anio`, el número de columnas aumentó a 33. También se observa que el conjunto de datos contiene variables de distintos tipos: numéricas, de texto, fecha, factor y lógicas.

```
> skim(df_raw)
— Data Summary —
Name      Values
Number of rows  df_raw
Number of columns  33

Column type frequency:
character      4
Date           1
factor         1
logical        9
numeric       18

Group variables      None

— Variable type: character —
skim_variable  n_missing complete_rate min max empty n_unique whitespace
1 Precip        0           1      1  7      0      540          0
2 PoorWeather  84803         0.288    1 10      0       38          0
3 PRCP         1932         0.984    1  5      0      540          0
4 TSHDSBRS GF  84803         0.288    1 10      0       38          0

— Variable type: Date —
skim_variable  n_missing complete_rate min      max      median      n_unique
1 Date          0           1 1940-01-01 1945-12-31 1944-06-26      2192

— Variable type: factor —
skim_variable  n_missing complete_rate ordered n_unique
1 Mes          0           1 TRUE          12
top_counts
1 dic: 11043, ago: 10547, oct: 10501, nov: 10408

— Variable type: logical —
skim_variable  n_missing complete_rate mean count
1 FT           119040          0 NaN  "": "
2 FB           119040          0 NaN  "": "
3 FTI          119040          0 NaN  "": "
4 ITH          119040          0 NaN  "": "
5 SD3          119040          0 NaN  "": "
6 RHX          119040          0 NaN  "": "
7 RHN          119040          0 NaN  "": "
8 RVG          119040          0 NaN  "": "
9 WTE          119040          0 NaN  "": "
```

En la primera parte del resumen se observa la estructura general del dataset. La variable `Date` ya fue transformada correctamente al tipo fecha, lo cual permite trabajar con información temporal. Además, la variable `Mes` aparece como factor con 12 categorías, una por cada mes del año.

También se confirma que algunas variables de tipo lógico, como FT, FB, FTI, ITH, SD3, RHX, RHN, RVG y WTE, tienen todos sus valores faltantes. Esto coincide con el análisis previo de valores nulos y refuerza la decisión de eliminarlas del dataset.

Variable type: numeric									
	skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	
1	STA	0	1	29659.	20953.	10001	11801	22508	
2	WindGustSpd	118508	0.00447	37.8	10.3	18.5	29.6	37.0	
3	MaxTemp	0	1	27.0	8.72	-33.3	25.6	29.4	
4	MinTemp	0	1	17.8	8.33	-38.3	15	21.1	
5	MeanTemp	0	1	22.4	8.30	-35.6	20.6	25.6	
6	Snowfall	1207	0.990	0.246	2.63	0	0	0	
7	YR	0	1	43.8	1.14	40	43	44	
8	MO	0	1	6.73	3.43	1	4	7	
9	DA	0	1	15.8	8.79	1	8	16	
10	DR	118507	0.00448	27.0	15.2	2	11	32	
11	SPD	118508	0.00447	20.4	5.56	10	16	20	
12	MAX	474	0.996	81.0	14.9	-28	78	85	
13	MIN	468	0.996	64.3	14.5	-37	59	70	
14	MEA	498	0.996	72.6	14.2	-32	69	78	
15	SNF	1207	0.990	0.00967	0.103	0	0	0	
16	SND	113477	0.0467	0	0	0	0	0	
17	PGT	118515	0.00441	12.1	5.73	0	8.5	11.6	
18	Anio	0	1	1944.	1.14	1940	1943	1944	
	p75	p100	hist						
1	33501	82506							
2	43.1	75.9							
3	31.7	50							
4	23.3	34.4							
5	27.2	40							
6	0	86.4							
7	45	45							
8	10	12							
9	23	31							
10	34	78							
11	23.2	41							
12	89	122							
13	74	94							
14	81	104							
15	0	3.4							
16	0	0							
17	15	23.9							
18	1945	1945							

En la sección de variables numéricas se muestran medidas como la media, desviación estándar, valores mínimos, máximos y percentiles. La variable objetivo MeanTemp tiene una media aproximada de 22.4 °C, una mediana de 25.6 °C y una desviación estándar de 8.30 °C, lo que indica una variabilidad considerable en la temperatura media diaria.

También se observa que variables como MaxTemp, MinTemp, YR, MO, DA y STA no presentan valores faltantes, por lo que son variables completas y potencialmente útiles para el análisis. En cambio, variables como WindGustSpd, DR, SPD, SND y PGT presentan una gran cantidad de valores faltantes, por lo que no son adecuadas para el modelado.

1.2 - Preprocesamiento para Modelado

En esta etapa se realizó la selección final de variables que serán utilizadas para construir los modelos de regresión. Primero se eliminaron columnas que no aportan información útil, principalmente aquellas con 100% de valores faltantes, variables con más del 95% de datos nulos y columnas redundantes.

También se eliminaron variables como MAX, MIN y MEA, ya que representan la misma información que MaxTemp, MinTemp y MeanTemp, pero en otra unidad de medida. En especial, MEA fue eliminada porque es equivalente a la variable objetivo MeanTemp, por lo que incluirla en el modelo podría generar fuga de información.

Posteriormente, las variables Precip y Snowfall fueron convertidas a formato numérico para que pudieran ser utilizadas correctamente en los modelos. En el caso de Snowfall, los valores faltantes fueron reemplazados por 0, asumiendo que la ausencia del dato puede representar ausencia de nieve registrada.

Después de aplicar la limpieza y eliminar registros incompletos, el dataset final de modelado quedó compuesto por 102,287 observaciones y 12 variables.

Las variables conservadas fueron:

Variable	Descripción
STA	Estación meteorológica
Date	Fecha del registro
Precip	Precipitación registrada
MaxTemp	Temperatura máxima diaria
MinTemp	Temperatura mínima diaria
MeanTemp	Temperatura media diaria (V.O)
Snowfall	Cantidad de nieve registrada
YR	Año
MO	Mes numérico
DA	Día
Mes	Mes como variable categórica
Anio	Año extraído de la fecha

1.3 - División Train/Test y Normalización

En esta etapa se preparó el dataset final para poder entrenar los modelos de regresión. Primero se definió una semilla con `set.seed(42)` para asegurar que la división de los datos sea reproducible, es decir, que los resultados puedan repetirse en futuras ejecuciones.

Posteriormente, se separaron las variables predictoras y la variable objetivo. En este caso, la variable objetivo es `MeanTemp`, ya que se busca predecir la temperatura media diaria. Las variables predictoras seleccionadas fueron `MaxTemp`, `MinTemp`, `Precip`, `Snowfall`, `MO`, `DA`, `YR` y `STA`, las cuales representan información climática, temporal y de estación meteorológica.

Después, el dataset se dividió en dos conjuntos: 80% para entrenamiento y 20% para prueba. El conjunto de entrenamiento quedó compuesto por 81,831 registros, mientras que el conjunto de prueba contiene 20,456 registros. Esta división permite entrenar los modelos con una parte de los datos y evaluar su desempeño con datos no vistos previamente.

También se aplicó un proceso de normalización mediante centrado y escalado. Esto transforma las variables para que tengan una escala similar, lo cual es importante para modelos sensibles a la magnitud de los datos, como regresión lineal regularizada, KNN, SVM o modelos que utilicen PCA.

Finalmente, se crearon los dataframes completos integrando las variables predictoras ya normalizadas junto con la variable objetivo `MeanTemp`. Estos conjuntos serán utilizados en la etapa de entrenamiento y evaluación de los modelos de regresión.

```
> set.seed(42)
>
> # Feature matrix y target
> features <- c("MaxTemp", "MinTemp", "Precip", "Snowfall", "MO", "DA", "YR", "STA")
> target <- "MeanTemp"
>
> X <- df_model %>% select(all_of(features))
> y <- df_model[[target]]
>
> # Split 80/20
> idx_train <- createDataPartition(y, p = 0.8, list = FALSE)
> X_train <- X[idx_train, ]; y_train <- y[idx_train]
> X_test <- X[-idx_train, ]; y_test <- y[-idx_train]
>
> cat("Train:", nrow(X_train), "| Test:", nrow(X_test), "\n")
Train: 81831 | Test: 20456
>
> # Normalización
> preproc <- preProcess(X_train, method = c("center", "scale"))
> X_train_sc <- predict(preproc, X_train)
> X_test_sc <- predict(preproc, X_test)
>
> # DataFrames completos para caret
> train_df <- cbind(X_train_sc, MeanTemp = y_train)
> test_df <- cbind(X_test_sc, MeanTemp = y_test)
>
```

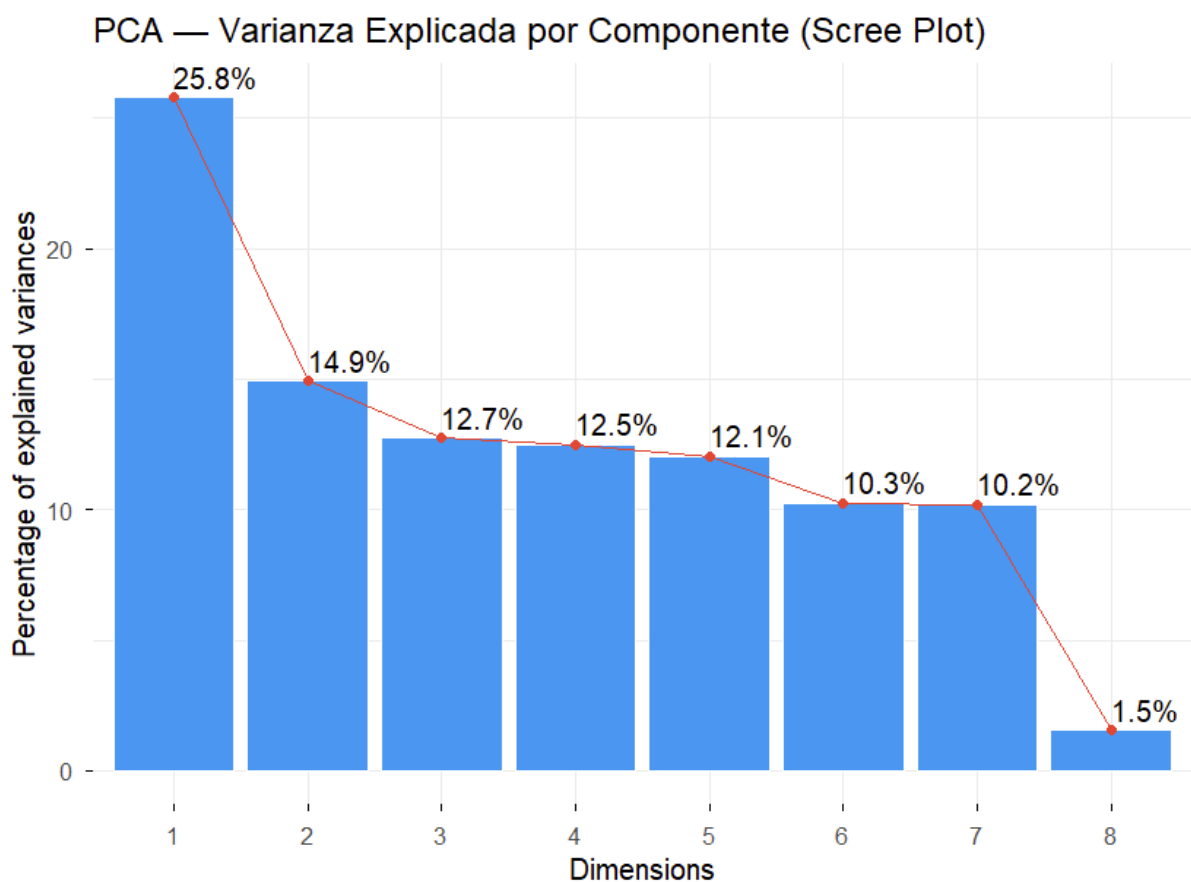
1.4 - Análisis de Componentes Principales (PCA)

En esta etapa se aplicó un PCA para analizar cómo se agrupan las variables predictoras y reducir la dimensionalidad del dataset. Este análisis permite identificar qué variables aportan más información a los primeros componentes principales y observar si existen relaciones importantes entre las variables utilizadas para el modelado.

El Scree Plot muestra el porcentaje de varianza que explica cada componente principal. En este caso, el primer componente (PC1) explica aproximadamente 25.8% de la variabilidad total, mientras que el segundo componente (PC2) explica 14.9%. En conjunto, los dos primeros componentes explican alrededor del 40.7% de la información del dataset.

Esto indica que la información no se concentra únicamente en uno o dos componentes, sino que está distribuida entre varias dimensiones. Por esta razón, aunque PC1 y PC2 permiten visualizar patrones generales, para el modelado puede ser necesario conservar más componentes si se desea retener mayor cantidad de información.

1.4.1 - Varianza por componente

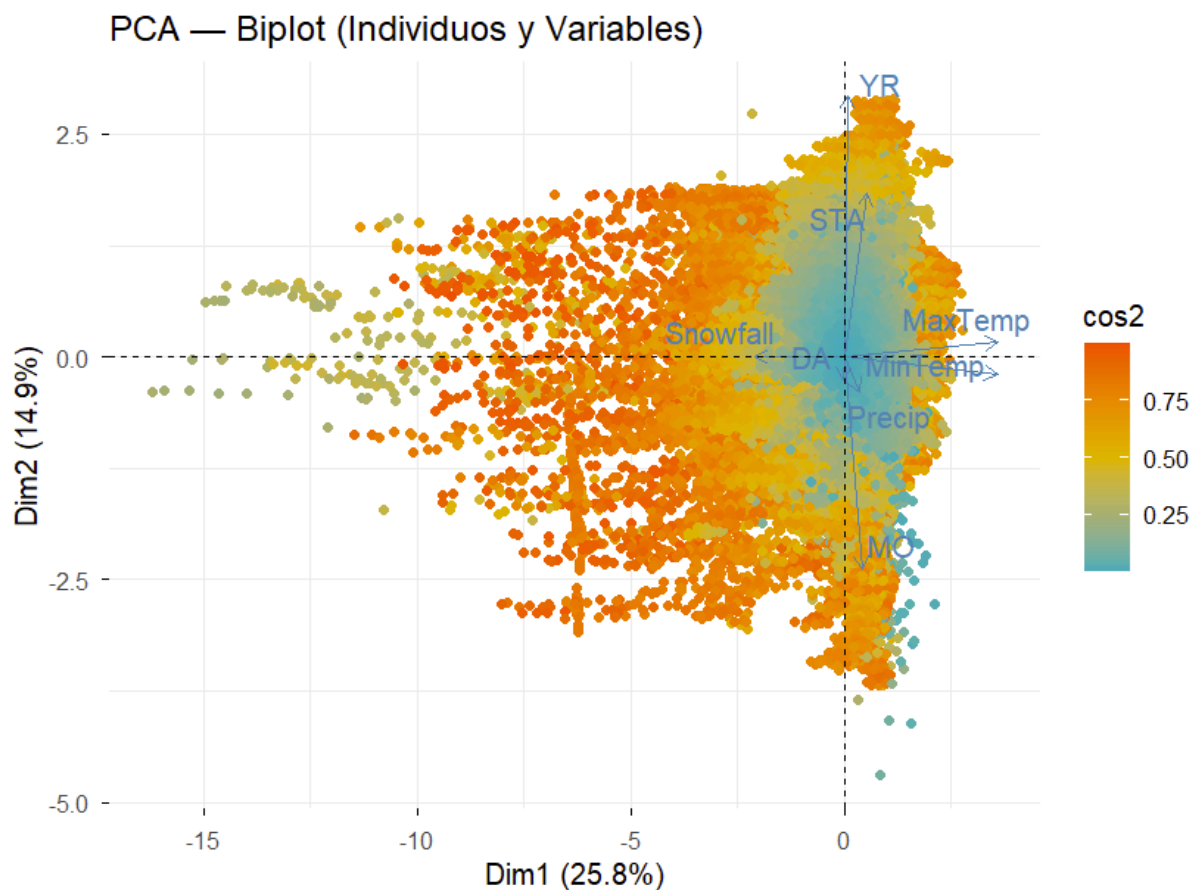


1.4.2 - Biplot de variables

El biplot permite observar tanto la distribución de las observaciones como la dirección e importancia de las variables dentro de los dos primeros componentes principales. En la gráfica se observa que MaxTemp y MinTemp apuntan en una dirección similar, lo cual confirma que están relacionadas entre sí y con la variación térmica del dataset.

También se aprecia que variables como YR, MO y STA tienen una influencia importante en la separación de los datos, especialmente sobre el segundo componente. Esto sugiere que el año, el mes y la estación meteorológica aportan información relevante para explicar diferencias entre observaciones.

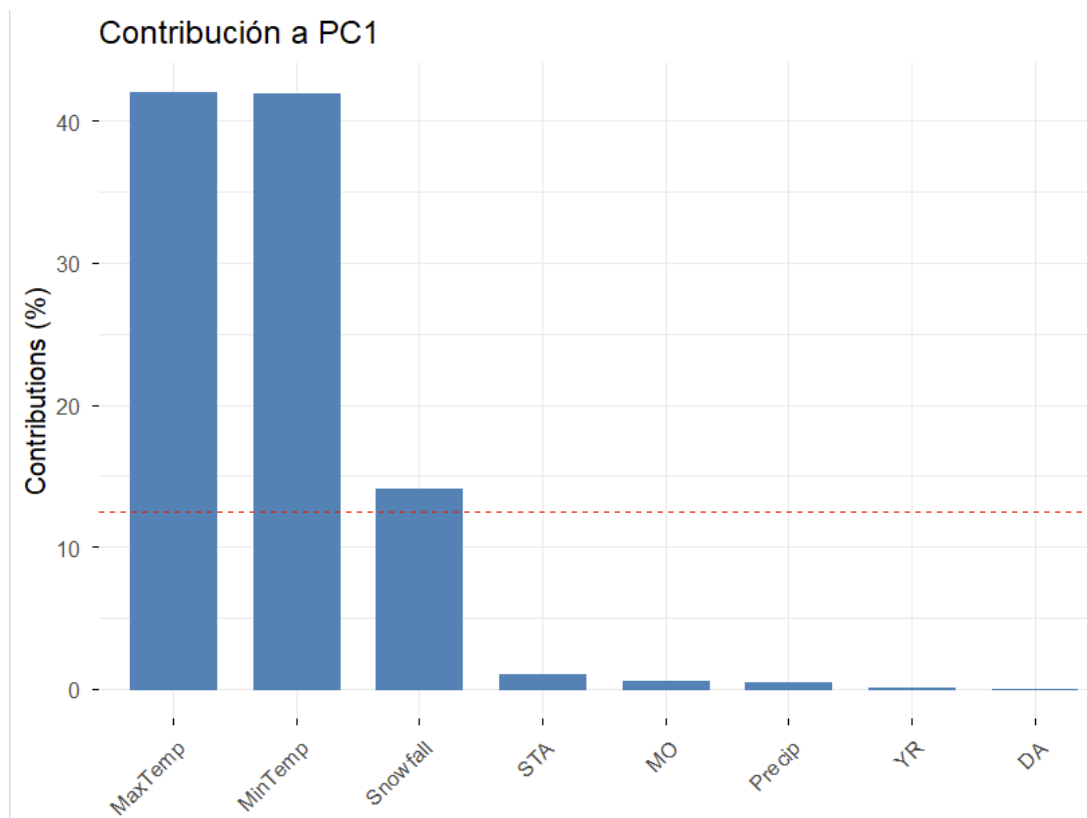
La variable Snowfall aparece separada de las variables de temperatura, lo que indica que representa un comportamiento distinto dentro del conjunto de datos.



1.4.3 - Gráfico de contribuciones a PC1 y PC2.

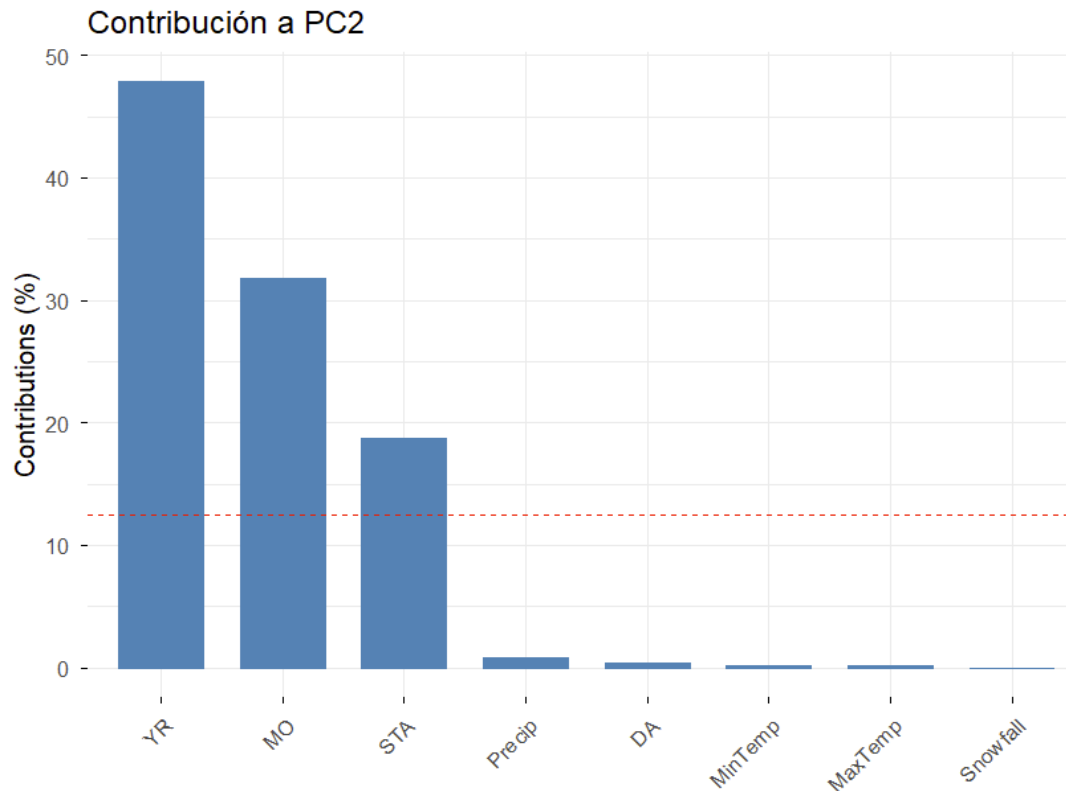
La gráfica de contribución a PC1 muestra qué variables aportan más información al primer componente principal. Se observa que MaxTemp y MinTemp son las variables con mayor contribución, ambas con valores superiores al 40%. Esto indica que el primer componente está principalmente relacionado con la variación de temperatura.

También aparece Snowfall con una contribución menor, pero todavía relevante. En cambio, variables como STA, MO, Precip, YR y DA tienen una contribución muy baja en este componente.



La contribución a PC2 muestra un patrón diferente. En este caso, las variables con mayor peso son YR, MO y STA. Esto indica que el segundo componente está más relacionado con factores temporales y de ubicación, como el año, el mes y la estación meteorológica.

Esto es importante porque complementa lo observado en PC1, mientras el primer componente resume principalmente la información de temperatura, el segundo componente captura diferencias asociadas al tiempo y a la estación.



1.5 - Función de Evaluación de Modelos

En esta etapa se creó una función reutilizable llamada `evaluar_modelo()`, cuyo objetivo es evaluar el desempeño de los modelos de regresión de forma uniforme. Esta función recibe el nombre del modelo, los valores reales de la variable objetivo y las predicciones generadas por el modelo.

```
> # Función reutilizable para evaluar regresión
> evaluar_modelo <- function(modelo_nombre, y_real, y_pred) {
+   rmse_val <- rmse(y_real, y_pred)
+   mae_val  <- mae(y_real, y_pred)
+   r2_val   <- 1 - sum((y_real - y_pred)^2) / sum((y_real - mean(y_real))^2)
+
+   cat("Modelo:", modelo_nombre, "\n")
+   cat("RMSE :", round(rmse_val, 4), "\n")
+   cat("MAE  :", round(mae_val, 4), "\n")
+   cat("R²   :", round(r2_val, 4), "\n")
+
+   data.frame(Modelo = modelo_nombre,
+               RMSE   = round(rmse_val, 4),
+               MAE     = round(mae_val, 4),
+               R2      = round(r2_val, 4))
+ }
>
> # Dataframe acumulador de resultados
> resultados <- data.frame()
>
```

Para medir el rendimiento se utilizaron tres métricas principales:

Métrica	Descripción
RMSE	Mide el error promedio del modelo, penalizando más los errores grandes. Mientras menor sea, mejor.
MAE	Representa el error absoluto promedio entre los valores reales y predichos. Mientras menor sea, mejor.
R ²	Indica qué proporción de la variabilidad de la variable objetivo es explicada por el modelo. Mientras más cercano a 1, mejor.

La función imprimirá los resultados en consola y también devuelve una tabla con las métricas calculadas. Esto permite guardar los resultados de cada modelo en un dataframe llamado resultados, facilitando la comparación posterior entre todos los modelos entrenados.

1.6 - Modelo Base 1: Regresión Lineal Múltiple

En esta etapa se entrenó un modelo de Regresión Lineal Múltiple utilizando las variables predictoras seleccionadas previamente. El objetivo del modelo fue predecir la temperatura media diaria MeanTemp a partir de variables como MaxTemp, MinTemp, Precip, Snowfall, MO, DA, YR y STA.

Para el entrenamiento se utilizó validación cruzada de 5 folds, lo cual permite evaluar el modelo de forma más robusta al dividir los datos de entrenamiento en diferentes subconjuntos. Posteriormente, el modelo fue evaluado sobre el conjunto de prueba para medir su capacidad de generalización.

1.6.1 - Resultados del modelo LM original

El resumen del modelo muestra que las variables MaxTemp y MinTemp tienen los coeficientes más altos y son estadísticamente significativas. Esto confirma lo observado en el análisis de correlación, donde ambas variables presentaron una relación muy fuerte con MeanTemp.

```

<
> summary(modelo_lm$finalModel)

Call:
lm(formula = .outcome ~ ., data = dat)

Residuals:
    Min       1Q   Median       3Q      Max
-44.161  -0.061   0.006   0.222  18.508

Coefficients:
              Estimate Std. Error  t value Pr(>|t|)
(Intercept)  22.504301   0.001885 11939.781 < 2e-16 ***
MaxTemp       4.329540   0.003922 1103.863 < 2e-16 ***
MinTemp       4.062903   0.003915 1037.863 < 2e-16 ***
Precip        0.005314   0.001926   2.758  0.00581 **
Snowfall      0.008959   0.001987   4.509 6.53e-06 ***
MO            -0.003367   0.001913  -1.760  0.07834 .
DA            -0.001423   0.001885  -0.755  0.45047
YR             0.001456   0.001929   0.755  0.45035
STA           -0.003293   0.001907  -1.727  0.08423 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

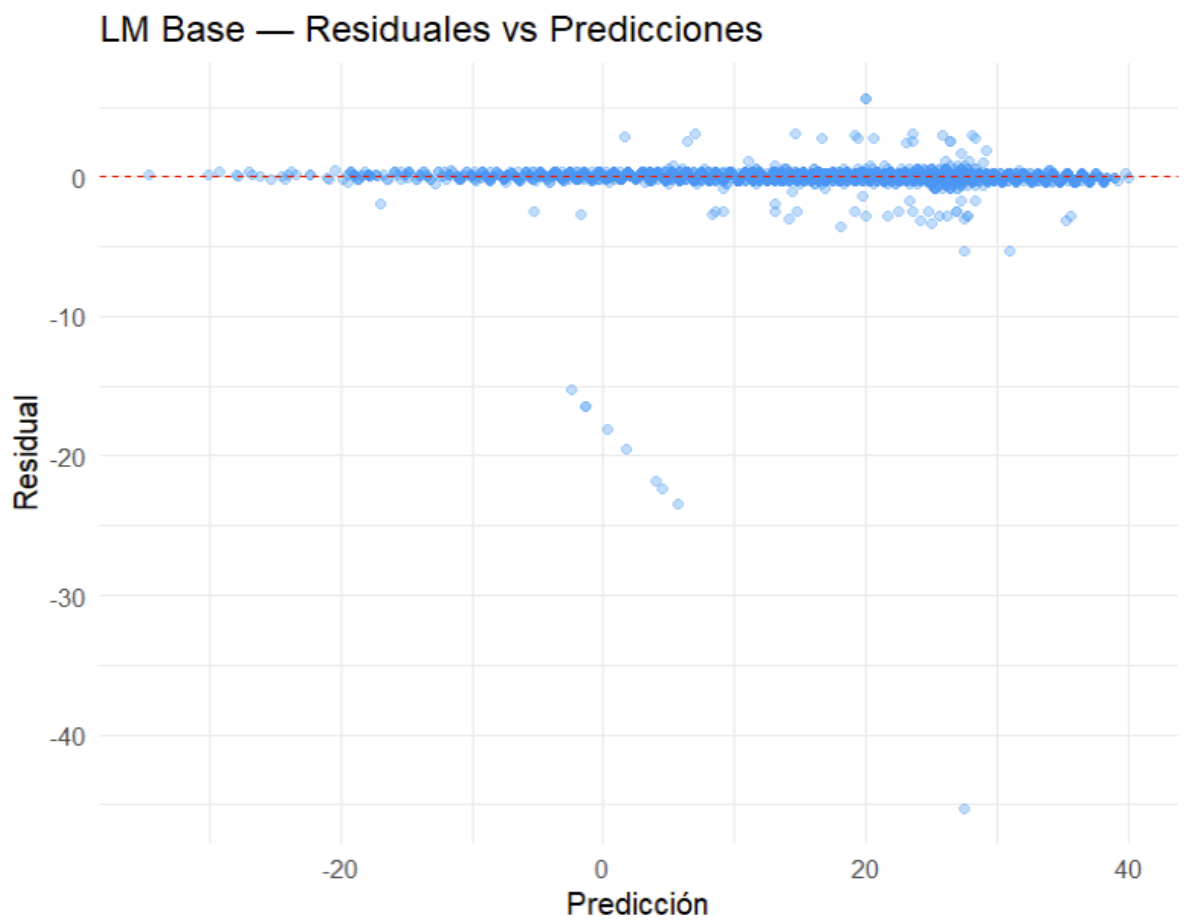
Residual standard error: 0.5392 on 81822 degrees of freedom
Multiple R-squared:  0.9956,    Adjusted R-squared:  0.9956
F-statistic: 2.313e+06 on 8 and 81822 DF,  p-value: < 2.2e-16

```

Estos resultados indican que el modelo tiene un desempeño muy alto. El valor de $R^2 = 0.9953$ significa que el modelo explica aproximadamente el 99.53% de la variabilidad de la temperatura media diaria. Además, los valores bajos de RMSE y MAE indican que el error promedio de predicción es reducido.

Modelo	RMSE	MAE	R^2
LM Original	0.5572	0.1676	0.9953

1.6.2 - Análisis de residuales



El gráfico de residuales permite observar la diferencia entre los valores reales y los valores predichos por el modelo. En general, la mayoría de los residuales se concentran cerca de la línea roja en cero, lo cual indica que las predicciones del modelo son bastante cercanas a los valores reales.

Sin embargo, también se observan algunos valores atípicos con residuales negativos muy marcados, lo que significa que en ciertos casos el modelo sobreestimó la temperatura real. A pesar de esto, la concentración principal de puntos alrededor de cero sugiere que el modelo lineal tiene un buen ajuste general.

1.6.4 - Comparación general

Al comparar ambos modelos, se observa que la Regresión Lineal original obtiene un desempeño ligeramente mejor que la versión con PCA. Esto puede deberse a que las variables originales, especialmente MaxTemp y MinTemp, ya contienen información muy directa y relevante para predecir MeanTemp.

El modelo de Regresión Lineal Múltiple obtuvo un rendimiento muy alto para la predicción de la temperatura media diaria. Aunque la versión con PCA también logró buenos resultados, el modelo original fue ligeramente superior, por lo que se considera más conveniente conservar las variables originales para este modelo base.

1.7 - Modelo Base 2: Support Vector Machine

En esta etapa se entrenó un modelo Support Vector Regression (SVR). Este modelo se utiliza para problemas de regresión y busca encontrar una función capaz de predecir valores continuos, en este caso la temperatura media diaria MeanTemp.

Debido al tamaño del dataset, se utilizó una muestra de 10,000 registros del conjunto de entrenamiento para reducir el tiempo de procesamiento y evitar problemas de memoria. Además, se aplicó validación cruzada de 3 folds, lo que permite evaluar el modelo de forma más estable durante el entrenamiento.

1.7.1 Resultados del modelo SVR original

El modelo SVR original obtuvo los siguientes resultados:

```
> # Support Vector Machine
> set.seed(42)
>
> set.seed(42)
> idx_samp <- sample(nrow(train_df), min(10000, nrow(train_df)))
> train_svm <- train_df[idx_samp, ]
>
> modelo_svr <- train(MeanTemp ~ .,
+                     data      = train_svm,
+                     method    = "svmRadial",
+                     trControl = trainControl(method = "cv", number = 3),
+                     preProcess = NULL)
>
> pred_svr <- predict(modelo_svr, newdata = test_df)
> res_svr <- evaluar_modelo("SVR - Original", y_test, pred_svr)
Modelo: SVR - Original
RMSE : 0.9479
MAE  : 0.3781
R2  : 0.9863
> resultados <- rbind(resultados, res_svr)
```

Estos resultados indican que el modelo tiene un buen desempeño general, ya que presenta un R^2 de 0.9863, lo que significa que explica aproximadamente el 98.63% de la variabilidad de la temperatura media diaria.

Sin embargo, al compararlo con la regresión lineal múltiple, sus errores son mayores, ya que el RMSE y el MAE aumentan. Esto puede deberse a que el modelo fue entrenado con una muestra reducida de los datos y no con el conjunto completo de entrenamiento.

1.7.2 - Modelo SVR con PCA

También se entrenó una versión del modelo SVR utilizando los componentes principales generados mediante PCA. El objetivo fue evaluar si la reducción de dimensionalidad podía mejorar el desempeño del modelo o hacerlo más eficiente.

El modelo SVR con PCA obtuvo los siguientes resultados:

```
> # Con PCA
> train_svm_pca <- train_pca_df[idx_samp, ]
>
> modelo_svr_pca <- train(MeanTemp ~ .,
+                          data      = train_svm_pca,
+                          method    = "svmRadial",
+                          trControl = trainControl(method = "cv", number = 3))
>
> pred_svr_pca <- predict(modelo_svr_pca, newdata = test_pca_df)
> res_svr_pca  <- evaluar_modelo("SVR - PCA", y_test, pred_svr_pca)
Modelo: SVR - PCA
RMSE : 0.9789
MAE  : 0.3807
R2  : 0.9854
> resultados  <- rbind(resultados, res_svr_pca)
>
```

Los resultados muestran que el modelo con PCA mantiene un desempeño alto, pero ligeramente inferior al modelo SVR original. El RMSE y el MAE son un poco mayores, mientras que el R^2 disminuye de 0.9863 a 0.9854.

1.7.3 - Comparación SVR original vs SVR con PCA

Al comparar ambos modelos, se observa que el SVR original obtuvo un rendimiento ligeramente mejor que el SVR con PCA. Esto indica que, para este caso, el uso de PCA no mejoró la capacidad predictiva del modelo.

Modelo	RMSE	MAE	R ²
SVR Original	0.9479	0.3781	0.9863
SVR PCA	0.9789	0.3807	0.9854

El modelo SVR logró un buen desempeño para predecir MeanTemp, aunque fue menos preciso que la regresión lineal múltiple. La versión con PCA no presentó mejora, por lo que en este modelo resulta más conveniente trabajar con las variables originales.

1.8 - Modelo Base 3: Regresión Penalizada — Ridge (Glmnet)

En esta etapa se entrenó un modelo de Ridge Regression, el cual es una variante de la regresión lineal que aplica regularización para reducir el efecto de variables altamente correlacionadas y evitar sobreajuste. En este caso se utilizó glmnet con $\alpha = 0$, lo que corresponde específicamente a Ridge.

El modelo fue entrenado usando validación cruzada y diferentes valores de lambda, que es el parámetro de regularización. Lambda controla qué tan fuerte será la penalización aplicada a los coeficientes del modelo.

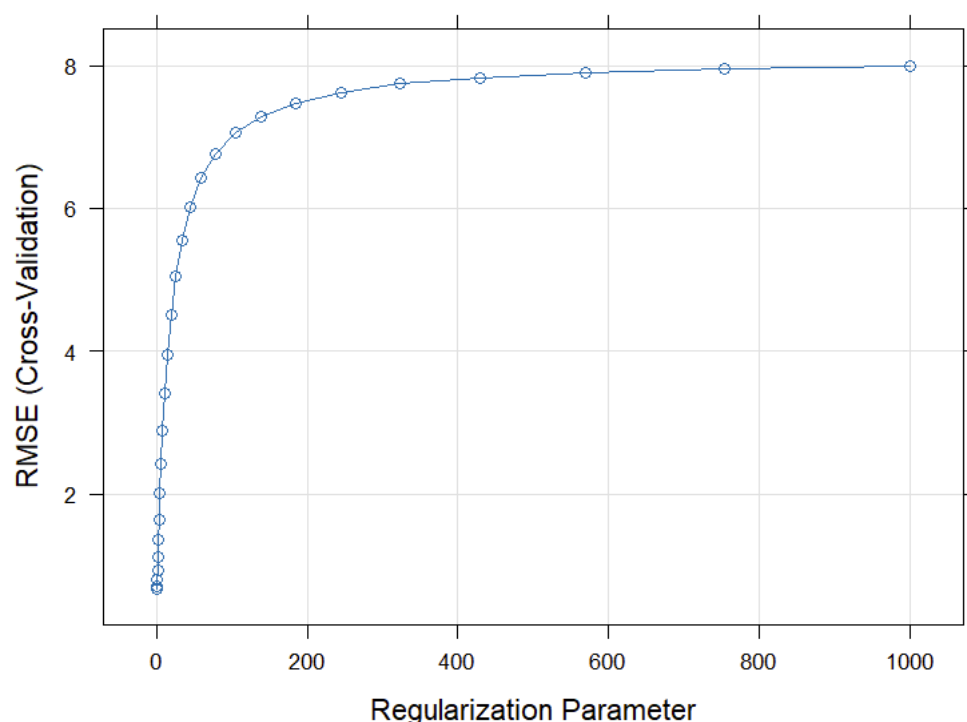
1.8.1 - Ridge Regression original

Los resultados obtenidos muestran que Ridge tiene un desempeño alto, con un R^2 de 0.9927, lo que indica que el modelo explica aproximadamente el 99.27% de la variabilidad de la temperatura media diaria. Sin embargo, sus errores son mayores que los obtenidos por la regresión lineal múltiple.

El modelo Ridge original obtuvo los siguientes resultados:

```
<
> # Ridge Regression (alpha = 0)
> set.seed(42)
>
> modelo_ridge <- train(MeanTemp ~ .,
+                       data      = train_df,
+                       method    = "glmnet",
+                       trControl = ctrl,
+                       tuneGrid  = expand.grid(alpha = 0,
+                                             lambda = 10^seq(-3, 3, length = 50)))
>
> # Lambda óptimo
> plot(modelo_ridge) # RMSE vs lambda
>
> pred_ridge <- predict(modelo_ridge, newdata = test_df)
> res_ridge <- evaluar_modelo("Ridge - Original", y_test, pred_ridge)
Modelo: Ridge - Original
RMSE : 0.6922
MAE  : 0.3442
R2  : 0.9927
> resultados <- rbind(resultados, res_ridge)
```

Por otro lado la gráfica de RMSE vs parámetro de regularización permite observar cómo cambia el error del modelo conforme aumenta el valor de lambda. En general, cuando la regularización aumenta demasiado, el RMSE también tiende a crecer, lo que indica una pérdida de precisión.



1.8.2 - Ridge Regression con PCA

También se entrenó una versión del modelo Ridge usando los componentes principales obtenidos mediante PCA. El objetivo fue evaluar si la reducción de dimensionalidad ayudaba a mejorar el desempeño del modelo.

El modelo Ridge con PCA obtuvo los siguientes resultados:

```
> # Con PCA
> modelo_ridge_pca <- train(MeanTemp ~ .,
+                           data      = train_pca_df,
+                           method    = "glmnet",
+                           trControl = ctrl,
+                           tuneGrid  = expand.grid(alpha = 0,
+                                                  lambda = 10^seq(-3, 3, length = 50)))
>
> pred_ridge_pca <- predict(modelo_ridge_pca, newdata = test_pca_df)
> res_ridge_pca  <- evaluar_modelo("Ridge - PCA", y_test, pred_ridge_pca)
Modelo: Ridge - PCA
RMSE : 0.9
MAE  : 0.5246
R2  : 0.9877
> resultados    <- rbind(resultados, res_ridge_pca)
```

Aunque el desempeño sigue siendo bueno, se observa que el modelo con PCA tuvo resultados inferiores al modelo Ridge original. El RMSE y el MAE aumentaron, mientras que el R^2 disminuyó.

1.8.3 - Comparación Ridge original vs Ridge con PCA

Al comparar ambos modelos, se observa que Ridge original obtuvo un mejor rendimiento que la versión con PCA. Esto indica que, para este caso, conservar las variables originales fue más conveniente que utilizar componentes principales.

Modelo	RMSE	MAE	R ²
Ridge Original	0.6922	0.3442	0.9927
Ridge PCA	0.9000	0.5246	0.9877

El modelo Ridge logró un buen rendimiento para predecir MeanTemp, aunque no superó a la regresión lineal múltiple. La versión con PCA presentó una disminución en el desempeño, por lo que en este caso la reducción de dimensionalidad no aportó una mejora al modelo.

1.9 -Modelo Base 4: KNN

En esta etapa se entrenó un modelo KNN Regressor como modelo base adicional. A diferencia de la regresión lineal, KNN no construye una ecuación explícita, sino que predice el valor de la variable objetivo a partir de los registros más similares dentro del conjunto de entrenamiento.

Para predecir MeanTemp, el modelo busca los k vecinos más cercanos y calcula un promedio de sus valores de temperatura media. Debido a que KNN se basa en distancias, fue necesario trabajar con variables normalizadas, ya que variables con escalas más grandes podrían influir de manera desproporcionada en el cálculo de cercanía.

También se entrenó una versión con PCA, con el objetivo de evaluar si la reducción de dimensionalidad ayudaba a mejorar el desempeño del modelo o reducir ruido en las variables predictoras.

1.9.1 - KNN Original

En esta etapa se entrenó un modelo KNN Regressor como modelo base adicional para predecir la temperatura media diaria MeanTemp. Este modelo funciona buscando los k vecinos más cercanos dentro del conjunto de entrenamiento y calculando el promedio de sus valores de temperatura para generar una predicción.

Debido a que KNN se basa en distancias, fue necesario trabajar con variables previamente normalizadas. Esto evita que variables con escalas más grandes tengan mayor influencia en el cálculo de cercanía.

Durante el entrenamiento se probaron distintos valores de k, y el mejor valor encontrado fue:

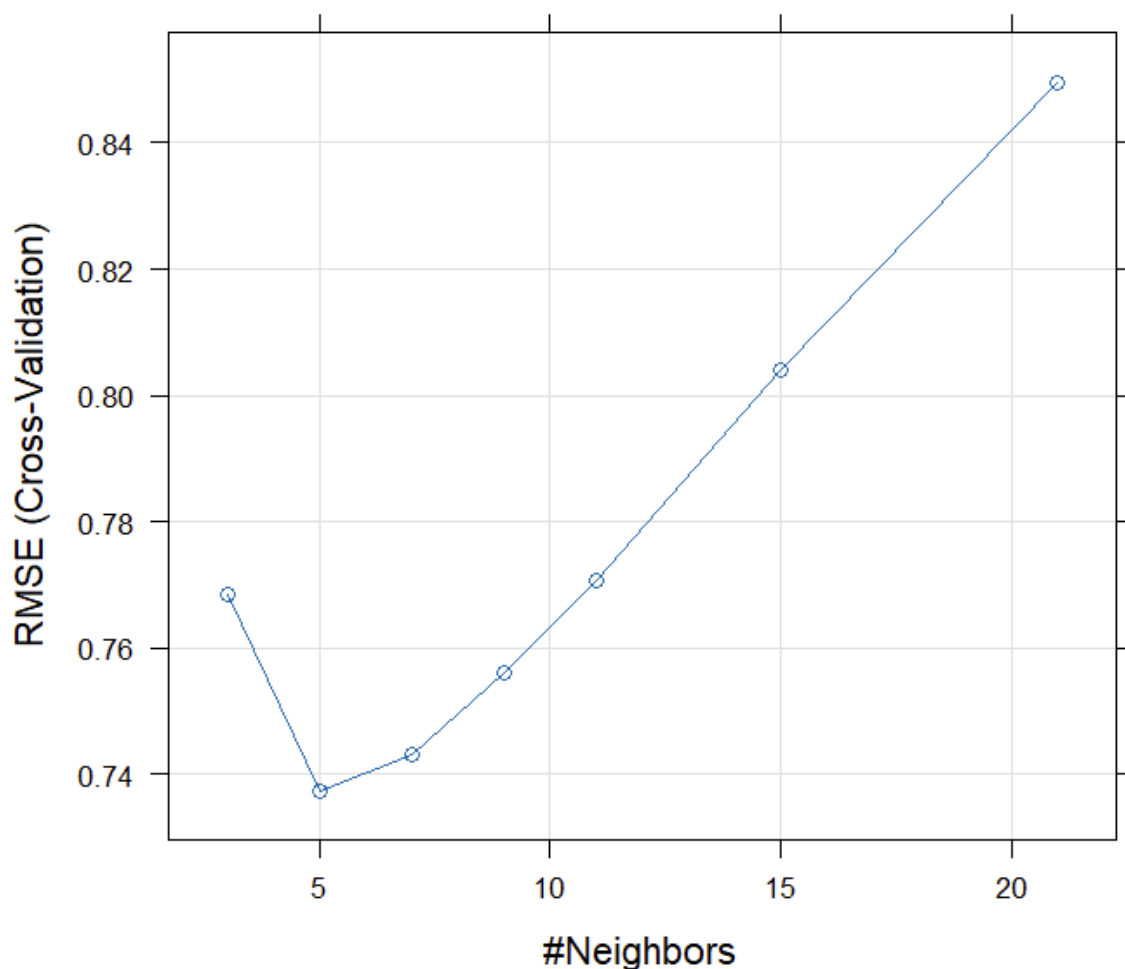
Parámetro	Valor
Mejor K	5

El modelo obtuvo los siguientes resultados:

```
> # Evaluación
> res_knn <- evaluar_modelo(
+   "KNN - Original",
+   y_test,
+   pred_knn
+ )
Modelo: KNN - Original
RMSE : 0.7206
MAE  : 0.4573
R2  : 0.9921
>
> resultados <- rbind(resultados, res_knn)
```

Estos resultados indican que el modelo KNN original tuvo un buen desempeño general, con un R^2 de 0.9921, lo que significa que explica aproximadamente el 99.21% de la variabilidad de la temperatura media diaria.

Sin embargo, al compararlo con la Regresión Lineal original, el KNN presentó errores más altos. Esto se observa en el RMSE de 0.7206 y el MAE de 0.4573, lo que indica que, aunque el modelo logra capturar patrones generales, sus predicciones tienen mayor error promedio.



La gráfica muestra el número de vecinos k , mientras que en el eje vertical aparece el RMSE obtenido mediante validación cruzada.

Se observa que el menor error se obtiene cuando $k = 5$, por lo que este fue seleccionado como el mejor valor para el modelo. Esto significa que el modelo obtuvo su mejor desempeño al calcular la predicción de MeanTemp usando los cinco registros más cercanos.

También se observa que, cuando el número de vecinos aumenta, el RMSE comienza a crecer. Esto indica que utilizar demasiados vecinos puede hacer que el modelo suavice

demasiado las predicciones, perdiendo precisión. Por lo tanto, para el KNN original, un valor pequeño de vecinos fue más adecuado.

1.9.2 - KNN con PCA

Después de entrenar el modelo KNN con las variables originales, se entrenó una segunda versión utilizando los componentes principales generados mediante PCA. El objetivo fue evaluar si la reducción de dimensionalidad ayudaba a mejorar el desempeño del modelo.

En esta versión, el modelo ya no utiliza directamente las variables originales, sino una representación reducida de los datos mediante componentes principales. Esto puede ayudar a disminuir la redundancia entre variables y concentrar la información más importante en menos dimensiones.

Durante el entrenamiento se probaron distintos valores de k , y el mejor valor encontrado fue 5, los resultados obtenidos fueron:

```
> # Evaluación
> res_knn_pca <- evaluar_modelo(
+   "KNN - PCA",
+   y_test,
+   pred_knn_pca
+ )
Modelo: KNN - PCA
RMSE : 0.6989
MAE  : 0.3569
R2  : 0.9926
>
> resultados <- rbind(resultados, res_knn_pca)
```

El modelo KNN con PCA obtuvo un RMSE de 0.6989, un MAE de 0.3569 y un R^2 de 0.9926. Estos resultados indican que el modelo mantiene un buen desempeño, ya que explica aproximadamente el 99.26% de la variabilidad de la temperatura media diaria.

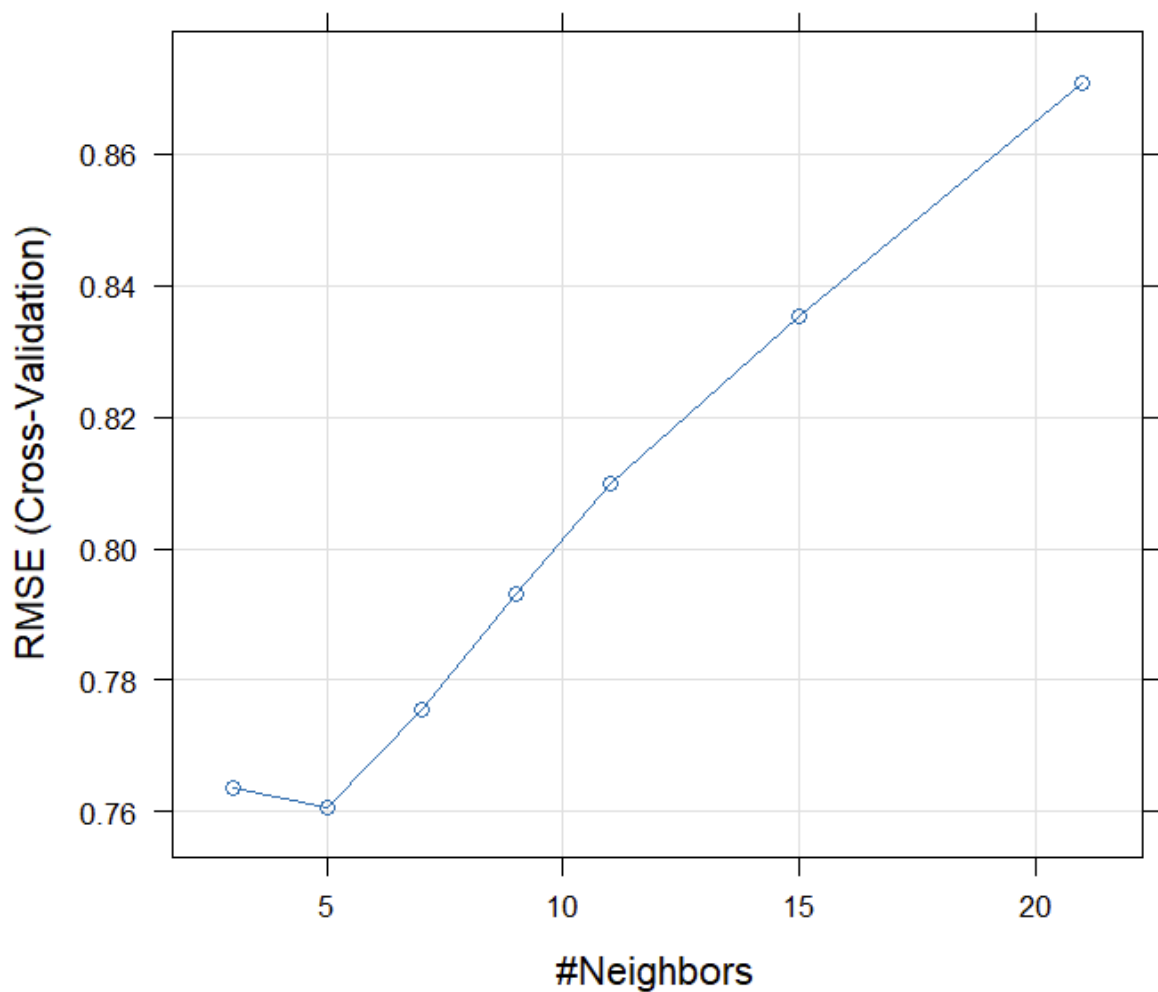
Además, al igual que en el KNN original, el mejor valor de k fue 5, lo que indica que el modelo funcionó mejor considerando pocos vecinos cercanos para calcular la predicción.

La gráfica del modelo KNN con PCA, donde en lugar de usar directamente las variables originales, se utilizaron los componentes principales generados mediante PCA.

En esta gráfica también se analiza cómo cambia el RMSE según el número de vecinos. Se observa que el error más bajo se encuentra nuevamente con valores bajos de k , especialmente alrededor de $k = 5$. A partir de ese punto, el RMSE aumenta conforme se incrementa el número de vecinos.

Este comportamiento indica que, aun después de reducir la dimensionalidad con PCA, el modelo KNN sigue funcionando mejor cuando considera pocos vecinos cercanos. Sin embargo, el RMSE general debe compararse con el KNN original para determinar si el uso de PCA realmente mejora el desempeño.

En general, esta gráfica muestra que el PCA no necesariamente mejora el comportamiento de KNN; más bien, conserva una tendencia similar, donde un número bajo de vecinos produce mejores resultados y valores altos de k reducen la precisión del modelo.



1.9.3 - Comparación KNN original vs KNN con PCA

Al comparar ambos modelos, se observa que KNN con PCA obtuvo un mejor rendimiento que la versión con original. Esto indica que, para este caso, usar los componentes principales fue más conveniente que usar las variables originales.

Modelo	RMSE	MAE	R ²	Mejor K
KNN Original	0.7206	0.4573	0.9921	5
KNN PCA	0.6989	0.3569	0.9926	5

Al comparar ambos modelos, se observa que KNN con PCA obtuvo un mejor desempeño que KNN original. El RMSE disminuyó de 0.7206 a 0.6989, lo que indica una reducción del error general del modelo. De igual manera, el MAE bajó de 0.4573 a 0.3569, por lo que el error absoluto promedio también fue menor.

Además, el valor de R² aumentó ligeramente, pasando de 0.9921 a 0.9926. Esto significa que la versión con PCA explicó una proporción ligeramente mayor de la variabilidad de MeanTemp.

En este caso, el uso de PCA sí aportó una mejora moderada al modelo KNN. Esto puede deberse a que KNN es sensible a la dimensionalidad y a la redundancia entre variables. Al transformar las variables originales en componentes principales, el modelo pudo trabajar con una representación más compacta y menos redundante de los datos.

Sin embargo, aunque el KNN con PCA mejoró al KNN original, ambos modelos quedaron por debajo de otros modelos más avanzados como XGBoost, Random Forest o GBM. Por lo tanto, KNN con PCA puede considerarse una mejora dentro de los modelos base, pero no el mejor modelo general del proyecto.

2.0 - Comparación de Modelos Base

Para evaluar el desempeño de los modelos base, se compararon las métricas obtenidas por cada algoritmo en sus versiones con variables originales y con PCA. Las métricas utilizadas fueron RMSE, MAE y R².

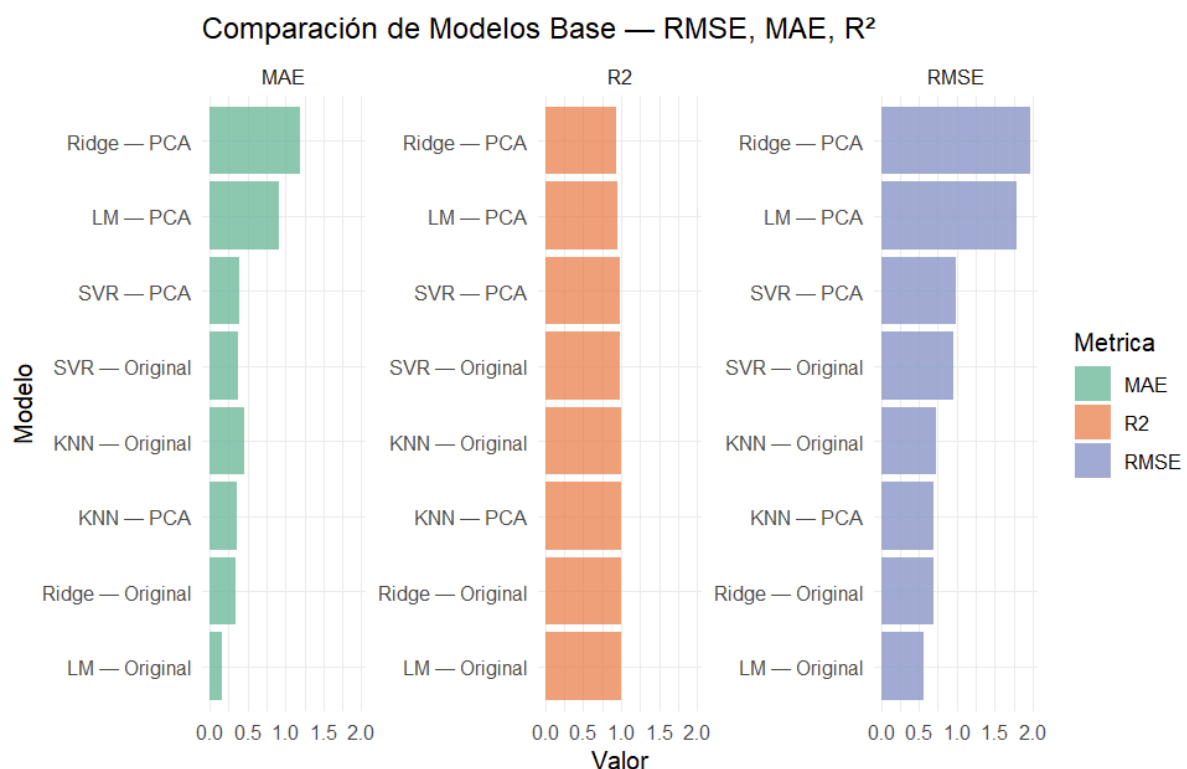
El RMSE y el MAE permiten medir el error de predicción del modelo; en ambos casos, valores más bajos indican un mejor rendimiento. Por otro lado, el R² indica la capacidad del modelo para explicar la variabilidad de la variable objetivo, por lo que valores cercanos a 1 representan un mejor ajuste.

En la comparación general se observa que la Regresión Lineal Múltiple original obtuvo el mejor desempeño entre los modelos base, ya que presentó valores bajos de error y un R^2 alto. Esto se debe a que las variables MaxTemp y MinTemp tienen una relación muy fuerte con la variable objetivo MeanTemp.

Los modelos SVR, Ridge y KNN también obtuvieron resultados favorables, aunque sus errores fueron mayores en comparación con la Regresión Lineal. En el caso de KNN, la versión con PCA presentó una mejora respecto al KNN original, ya que redujo el RMSE de 0.7206 a 0.6989 y aumentó ligeramente el R^2 de 0.9921 a 0.9926. Esto indica que, para KNN, la reducción de dimensionalidad ayudó a trabajar con una representación más compacta de los datos.

Sin embargo, en los demás modelos base, el uso de PCA no generó una mejora clara. En particular, las versiones con PCA de Ridge, SVR y Regresión Lineal tuvieron un desempeño igual o inferior a sus versiones originales. Esto sugiere que, aunque PCA puede ser útil para algunos modelos como KNN, no necesariamente mejora todos los algoritmos.

En conclusión, para esta primera etapa de modelado, la Regresión Lineal Múltiple original se mantiene como el modelo base con mejor desempeño general para predecir MeanTemp. No obstante, el modelo KNN con PCA mostró una mejora respecto a su versión original, lo que evidencia que el efecto del PCA depende del tipo de algoritmo utilizado.



Parte 2 - Mejora de Modelos

2.1 - Ingeniería de Características

En esta segunda parte del proyecto se aplicó ingeniería de características con el objetivo de enriquecer el dataset y mejorar el desempeño de los modelos de regresión. A partir de las variables originales se crearon nuevas variables derivadas que representan mejor el comportamiento térmico, temporal y estacional de los datos.

Después de aplicar este proceso, el dataset quedó conformado por 102,287 observaciones y 17 variables. Las nuevas características generadas fueron TempRange, Estacion_num, EsCalorosoMax, LogPrecip, InteracTemp, MO_sin, MO_cos, DA_sin y DA_cos.

```
> head(df_fe, 5)
# A tibble: 5 x 17
  STA Date      Precip MaxTemp MinTemp MeanTemp Mes   Anio TempRange Estacion_num
  <dbl> <date>      <dbl>   <dbl>   <dbl>   <dbl> <ord> <dbl>   <dbl>
1 10001 1942-07-01  1.02    25.6    22.2    23.9 jul   1942    3.33      3
2 10001 1942-07-02  0      28.9    21.7    25.6 jul   1942    7.22      3
3 10001 1942-07-03  2.54    26.1    22.2    24.4 jul   1942    3.89      3
4 10001 1942-07-04  2.54    26.7    22.2    24.4 jul   1942    4.44      3
5 10001 1942-07-05  0      26.7    21.7    24.4 jul   1942    5        3
# i 7 more variables: EsCalorosoMax <int>, LogPrecip <dbl>, InteracTemp <dbl>, MO_sin <dbl>,
#   MO_cos <dbl>, DA_sin <dbl>, DA_cos <dbl>
> |
```

La variable TempRange representa el rango térmico diario, calculado como la diferencia entre MaxTemp y MinTemp. Esta variable permite conocer qué tanta variación de temperatura hubo durante el día.

También se creó Estacion_num, que codifica la estación del año según el mes del registro. Esta variable permite incorporar información estacional al modelo, ya que anteriormente se observó que la temperatura media cambia dependiendo del mes.

La variable EsCalorosoMax indica si la temperatura máxima superó los 30 °C, permitiendo identificar días con temperaturas altas. Por otro lado, LogPrecip representa la precipitación transformada mediante logaritmo, con el propósito de reducir el impacto de valores extremos.

Además, se creó InteracTemp, que representa la interacción entre MaxTemp y MinTemp, debido a que ambas variables tienen una relación muy fuerte con MeanTemp. Finalmente, se generaron variables cíclicas para el mes y el día mediante seno y coseno, lo cual permite representar mejor la naturaleza repetitiva del tiempo.

Esto lo que permitió fue generar variables más representativas para los modelos, incorporando información térmica, estacional y temporal. Estas nuevas características serán utilizadas para evaluar si los modelos mejorados logran un mejor desempeño en comparación con los modelos base.

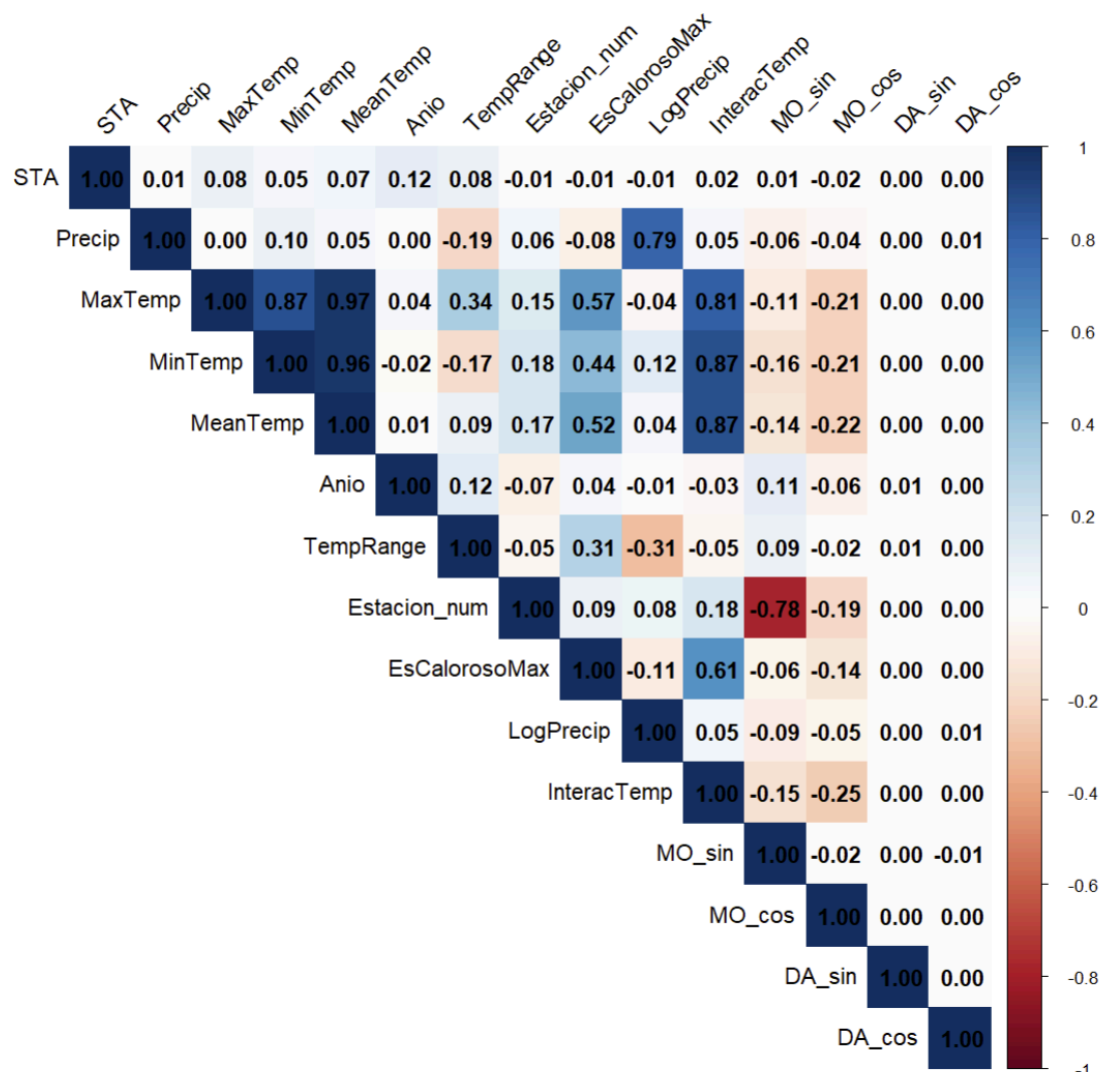
2.2 - Análisis Post-FE y Correlación

Después de aplicar la ingeniería de características, se generó una nueva matriz de correlación para evaluar la relación entre las variables originales, las variables derivadas y la variable objetivo MeanTemp.

Los resultados muestran que MaxTemp y MinTemp continúan siendo las variables más relacionadas con MeanTemp, con correlaciones aproximadas de 0.97 y 0.96. Además, la nueva variable InteracTemp, creada a partir de la interacción entre ambas temperaturas, presenta una correlación alta de aproximadamente 0.87, por lo que puede aportar información relevante al modelo.

También se observa que EsCalorosoMax tiene una correlación moderada con MeanTemp, cercana a 0.52, lo cual indica que puede ayudar a identificar días con temperaturas altas. Por otro lado, variables como TempRange y las variables cíclicas MO_sin, MO_cos, DA_sin y DA_cos presentan correlaciones más bajas, aunque pueden aportar información temporal y estacional.

Finalmente, LogPrecip mantiene una alta relación con Precip, lo cual es esperado al tratarse de una transformación de la misma variable. En general, esta matriz permite justificar la inclusión de nuevas características y detectar posibles redundancias antes del modelado mejorado.



2.3 - Split y Normalización Post FE

Después de crear las nuevas variables derivadas, se realizó nuevamente la preparación del dataset para el entrenamiento de los modelos mejorados. Primero se separaron las variables predictoras de la variable objetivo MeanTemp, ya que esta continúa siendo el valor que se busca predecir.

Posteriormente, el conjunto de datos se dividió en 80% para entrenamiento y 20% para prueba, manteniendo la misma estrategia utilizada en la primera parte del proyecto. Esto permite comparar de manera más justa los modelos base con los modelos mejorados.

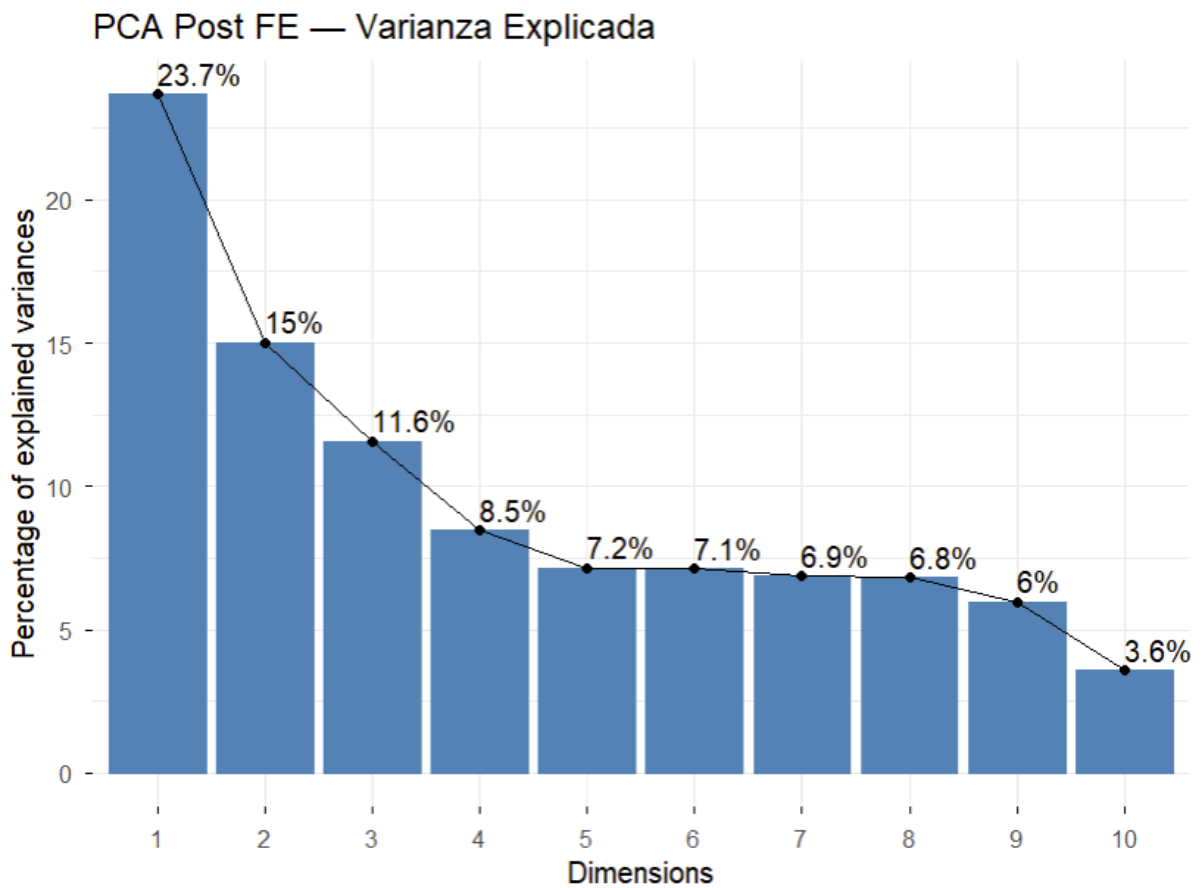
También se aplicó un proceso de normalización mediante centrado y escalado a las variables predictoras. Esta transformación es importante porque las nuevas variables tienen escalas diferentes; por ejemplo, InteracTemp puede tener valores mucho más grandes que variables como MO_sin o DA_cos. Al normalizar los datos, se evita que una variable domine sobre las demás únicamente por su magnitud.

Finalmente, se construyeron los nuevos conjuntos train_fe_df y test_fe_df, los cuales contienen las variables normalizadas junto con la variable objetivo MeanTemp. Estos datasets serán utilizados para entrenar y evaluar los modelos mejorados.

```
> set.seed(42)
> features_fe <- setdiff(names(df_fe), "MeanTemp")
>
> X_fe <- df_fe %>% select(all_of(features_fe))
> y_fe <- df_fe$MeanTemp
>
> idx_tr <- createDataPartition(y_fe, p = 0.8, list = FALSE)
> X_tr <- X_fe[idx_tr, ]; y_tr <- y_fe[idx_tr]
> X_te <- X_fe[-idx_tr, ]; y_te <- y_fe[-idx_tr]
>
> preproc_fe <- preProcess(X_tr, method = c("center", "scale"))
> X_tr_sc <- predict(preproc_fe, X_tr)
> X_te_sc <- predict(preproc_fe, X_te)
>
> train_fe_df <- cbind(X_tr_sc, MeanTemp = y_tr)
> test_fe_df <- cbind(X_te_sc, MeanTemp = y_te)
> |
```

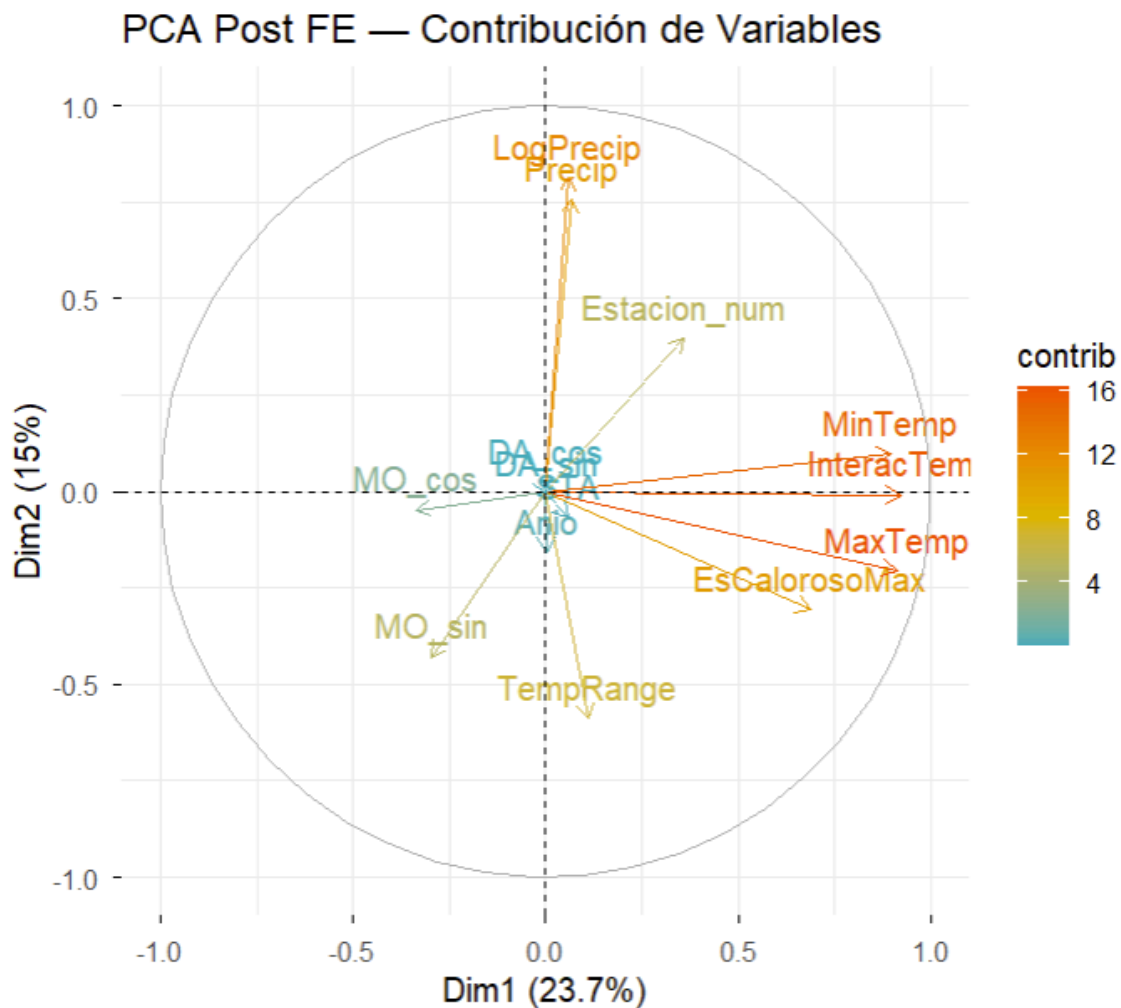
2.4 - PCA Post Feature Engineering

Después de aplicar la ingeniería de características, se realizó nuevamente un Análisis de Componentes Principales (PCA) con el objetivo de observar cómo se distribuye la información de las nuevas variables y determinar cuántos componentes son necesarios para conservar una proporción importante de la varianza del dataset.



En la gráfica de varianza explicada se observa que el primer componente principal (Dim1) explica aproximadamente 23.7% de la variabilidad total, mientras que el segundo componente (Dim2) explica alrededor de 15%. En conjunto, los dos primeros componentes explican cerca del 38.7% de la información.

Esto indica que, aunque los primeros componentes concentran una parte importante de la variabilidad, la información sigue distribuida entre varios componentes. Por esta razón, se calculó el número de componentes necesarios para conservar al menos el 90% de la varianza, con el fin de utilizar una representación reducida pero suficientemente informativa para los modelos con PCA.



La gráfica de contribución de variables permite identificar qué características tienen mayor influencia en los primeros componentes principales. Se observa que variables como MaxTemp, MinTemp e InteracTemp tienen una fuerte contribución sobre el primer componente, lo cual confirma que la temperatura sigue siendo el factor principal que explica la variabilidad del dataset.

Por otro lado, variables como Precip y LogPrecip tienen una contribución importante en el segundo componente, lo que indica que la precipitación aporta información distinta a la relacionada directamente con la temperatura. También se observa que variables como MO_sin, MO_cos, Estacion_num y TempRange ayudan a representar patrones temporales, estacionales y de variabilidad térmica.

2.5 - Ajuste de Hiperparámetros — Modelo Mejorado 1: Ridge/Lasso

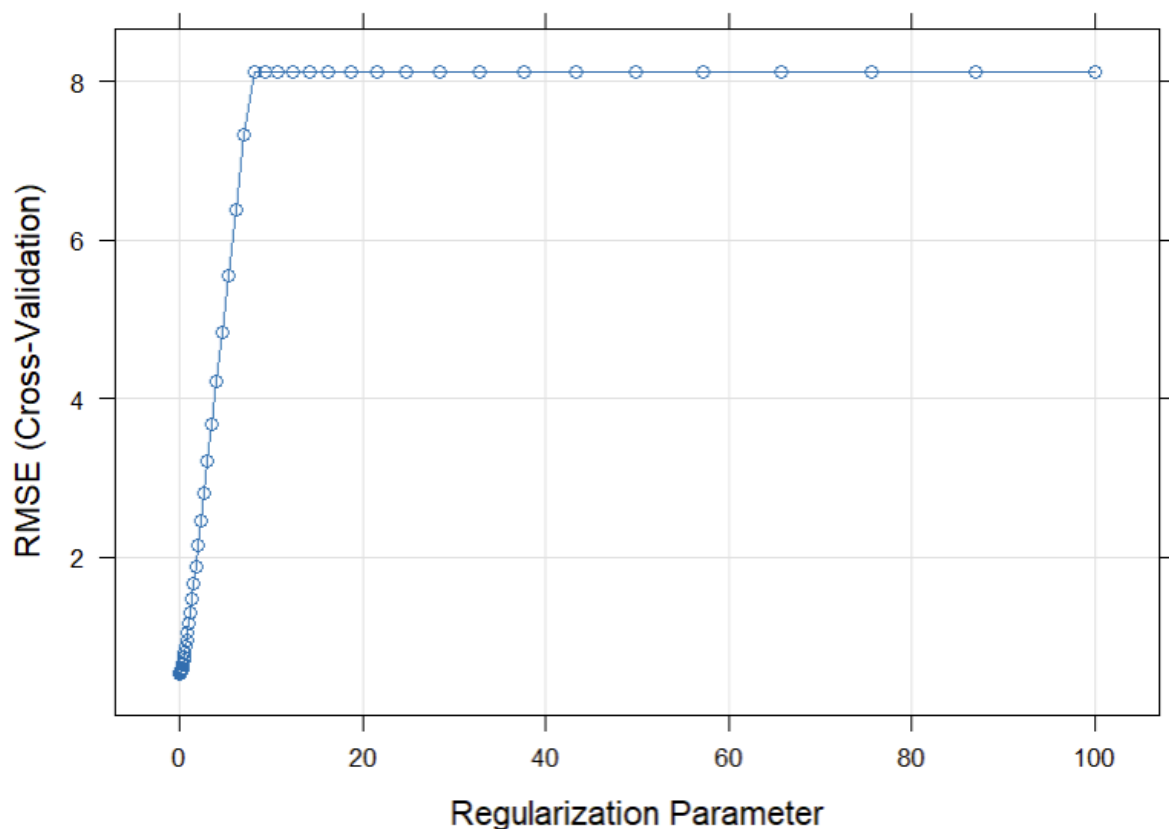
2.5.1 - Lasso con Feature Engineering

Después de aplicar ingeniería de características, se entrenaron modelos de regresión penalizada con el objetivo de evaluar si las nuevas variables mejoraron el desempeño predictivo. Para esta etapa se utilizaron modelos Lasso y Elastic Net, ambos implementados con glmnet, aplicando validación cruzada de 10 folds.

Estos modelos utilizan un parámetro de regularización llamado lambda, el cual controla qué tanto se penalizan los coeficientes del modelo. En el caso de Lasso, esta penalización también puede ayudar a reducir la influencia de variables poco relevantes.

Modelo	RMSE	MAE	R ²
Lasso - FE	0.5592	0.1710	0.9952

El modelo Lasso con Feature Engineering obtuvo un desempeño alto, con un RMSE de 0.5592, un MAE de 0.1710 y un R² de 0.9952. Esto indica que el modelo logra explicar aproximadamente el 99.52% de la variabilidad de la temperatura media diaria. Sin embargo, aunque el rendimiento es muy bueno, no supera ligeramente al modelo de Regresión Lineal original.



2.5.2 - Elastic Net con Feature Engineering

El modelo Elastic Net con Feature Engineering también presentó un desempeño alto, con un R² de 0.9952. No obstante, sus errores fueron ligeramente mayores que los de Lasso, ya

que obtuvo un RMSE de 0.5593 y un MAE de 0.1768. Esto indica que Elastic Net mantiene una buena capacidad predictiva, pero no representa una mejora frente a Lasso ni frente a la Regresión Lineal original.

```

Mejor Elastic Net -> Alpha: 0.2 | Lambda: 0.04941713
>
> pred_en <- predict(modelo_en, newdata = test_fe_numeric)
> res_en <- evaluar_modelo("Elastic Net - FE", y_te, pred_en)
Modelo: Elastic Net - FE
RMSE : 0.5593
MAE : 0.1768
R² : 0.9952
> resultados <- rbind(resultados, res_en)

```

2.5.3 - Lasso con Feature Engineering + PCA

El modelo Lasso con Feature Engineering + PCA presentó una disminución importante en el desempeño. Su RMSE aumentó a 2.2023, el MAE subió a 1.5714 y el R² bajó a 0.9261. Esto indica que, en este caso, aplicar PCA después de la ingeniería de características no ayudó al modelo, ya que se perdió información relevante para predecir MeanTemp.

```

> res_lasso_pca <- evaluar_modelo("Lasso - FE + PCA",
Modelo: Lasso - FE + PCA
RMSE : 2.2023
MAE : 1.5714
R² : 0.9261
> resultados <- rbind(resultados, res_lasso_pca)
>

```

2.5.4 - Comparación de modelos mejorados

Al comparar los modelos mejorados, se observa que Lasso FE obtuvo el mejor desempeño dentro de esta etapa, ya que presentó el menor RMSE y MAE. Elastic Net FE tuvo resultados muy similares, aunque con un error ligeramente mayor. En cambio, Lasso FE + PCA fue el modelo con menor rendimiento, por lo que se concluye que el uso de PCA no mejoró los resultados después de la ingeniería de características.

```

> # Tabla Comparativa Final
> print(resultados)

```

	Modelo	RMSE	MAE	R2
1	LM - Original	0.5572	0.1676	0.9953
2	LM - PCA	0.5592	0.1782	0.9952
3	SVR - Original	0.9479	0.3781	0.9863
4	SVR - PCA	0.9789	0.3807	0.9854
5	Ridge - Original	0.6922	0.3442	0.9927
6	Ridge - PCA	0.9000	0.5246	0.9877
7	Lasso - FE	0.5592	0.1710	0.9952
8	Lasso - FE	0.5592	0.1710	0.9952
9	Elastic Net - FE	0.5593	0.1768	0.9952
10	Lasso - FE	0.5592	0.1710	0.9952
11	Elastic Net - FE	0.5593	0.1768	0.9952
12	Lasso - FE + PCA	2.2023	1.5714	0.9261

```

>

```

2.6 - Modelo Mejorado 2: SVR con Tuning

En esta etapa se entrenó una versión mejorada del modelo Support Vector Regression (SVR) utilizando las variables generadas mediante ingeniería de características. El objetivo fue evaluar si el ajuste de hiperparámetros podría mejorar el desempeño obtenido por el modelo SVR base.

Debido al tamaño del dataset, se utilizó una muestra de 15,000 registros del conjunto de entrenamiento para reducir el tiempo de procesamiento. Posteriormente, el modelo fue entrenado con validación cruzada de 5 folds.

Para mejorar el modelo se aplicó un Grid Search, probando diferentes combinaciones de los hiperparámetros C y sigma:

Hiperparámetro	Descripción
C	Controla el nivel de penalización por errores. Valores altos buscan ajustar mejor los datos, pero pueden aumentar el riesgo de sobreajuste.
Sigma	Controla la influencia del kernel radial. Define qué tan flexible será la frontera de decisión del modelo.

2.6.1 - SVR con Feature Engineering + Tuning

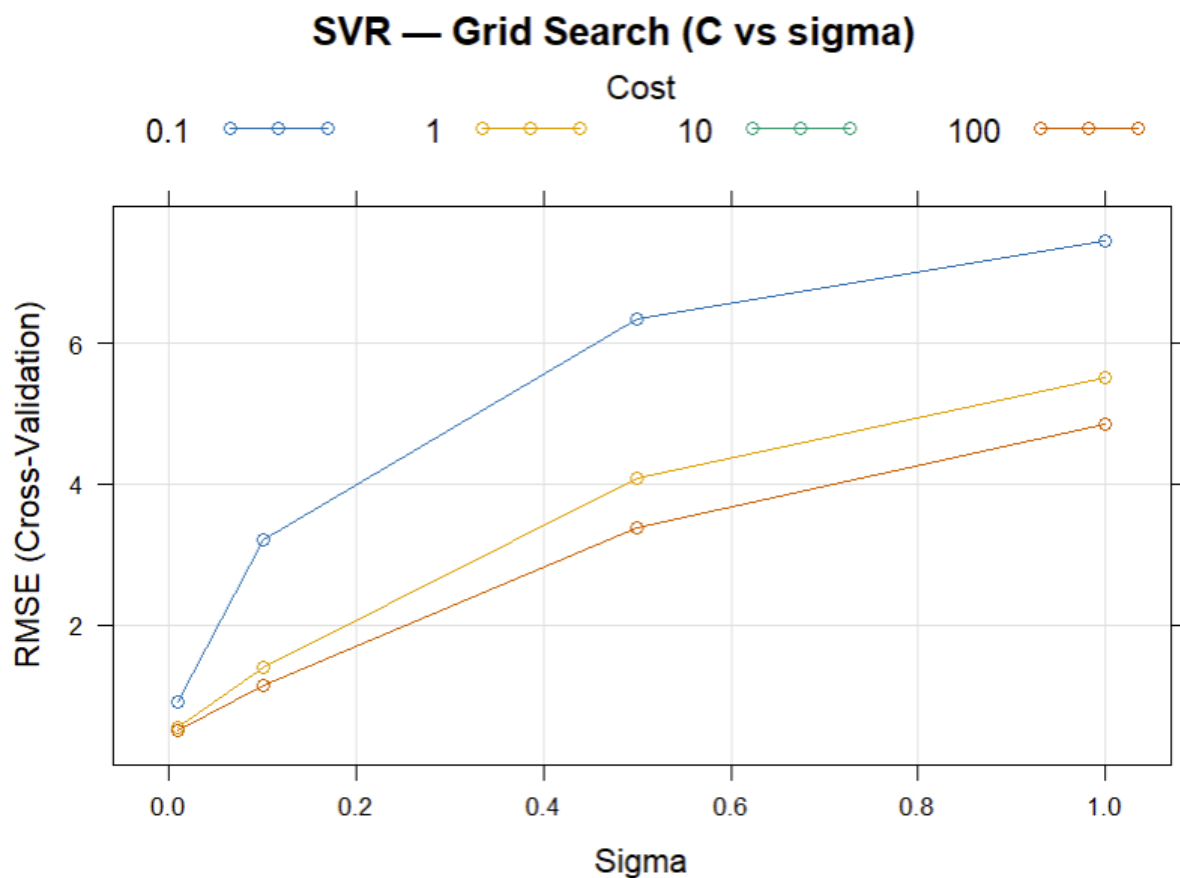
El modelo SVR mejorado fue entrenado utilizando las nuevas variables generadas mediante ingeniería de características y ajuste de hiperparámetros. A través del Grid Search se evaluaron diferentes combinaciones de C y sigma, seleccionando aquella que obtuvo el mejor rendimiento durante la validación cruzada. Este ajuste permite que el modelo encuentre una configuración más adecuada para capturar relaciones no lineales entre las variables predictoras y la temperatura media diaria.

El modelo obtuvo los siguientes resultados:

Modelo	RMSE	MAE	R ²
SVR — FE + Tuning	0.5145	0.2963	0.9960

Estos resultados muestran una mejora respecto al SVR base, ya que el RMSE disminuyó de 0.9479 a 0.5145 y el R² aumentó de 0.9863 a 0.9960. Esto indica que la combinación de ingeniería de características y ajuste de hiperparámetros mejoró la capacidad predictiva del modelo.

La gráfica de *Grid Search* muestra cómo cambia el RMSE según los valores de C y sigma. Se observa que los valores bajos de sigma generan mejores resultados, mientras que valores más altos incrementan el error del modelo.



El modelo SVR con Feature Engineering + Tuning presentó un desempeño superior al SVR original. La optimización de hiperparámetros permitió mejorar la precisión del modelo, convirtiéndose en uno de los modelos con mejor rendimiento hasta esta etapa.

2.6.2 - SVR con Feature Engineering + PCA + Tuning

También se entrenó una versión del modelo SVR utilizando PCA después de la ingeniería de características. Esta variante busca reducir la dimensionalidad del dataset y disminuir la redundancia entre variables. Sin embargo, su desempeño debe compararse con el modelo SVR sin PCA para determinar si la reducción de dimensionalidad conserva suficiente información para la predicción de MeanTemp.

El modelo fue entrenado usando validación cruzada de 5 folds y el mismo esquema de búsqueda de hiperparámetros utilizado en el SVR con Feature Engineering. Posteriormente, se evaluó sobre el conjunto de pruebas.

Los resultados obtenidos fueron los siguientes:

Modelo	RMSE	MAE	R ²
SVR — FE + PCA + Tuning	0.7072	0.5870	0.9924

El modelo obtuvo un R² de 0.9924, lo que indica que todavía mantiene una alta capacidad para explicar la variabilidad de la temperatura media diaria. Sin embargo, al compararlo con el modelo SVR — FE + Tuning sin PCA, su desempeño fue menor, ya que el RMSE aumentó de 0.5145 a 0.7071 y el MAE aumentó de 0.2963 a 0.4870.

Esto indica que, aunque PCA permitió reducir la dimensionalidad, también pudo haber eliminado parte de la información relevante para predecir MeanTemp. Por lo tanto, para este modelo, la versión con variables originales después de la ingeniería de características fue más efectiva que la versión con PCA.

El modelo SVR con Feature Engineering + PCA + Tuning presentó un buen rendimiento general, pero no superó al SVR mejorado sin PCA. Por ello, se concluye que en este caso el uso de PCA no aportó una mejora al modelo SVR.

2.7 - Modelo Mejorado 3: Regresión Lineal con Interacciones

En esta etapa se entrenó un modelo de Regresión Lineal mejorado utilizando las variables obtenidas después de la ingeniería de características. Además, se agregaron términos cuadráticos para MaxTemp y MinTemp, creando las variables MaxTemp² y MinTemp².

El objetivo de agregar estos términos polinomiales fue permitir que el modelo capturara posibles relaciones no lineales entre las temperaturas máxima, mínima y la variable objetivo MeanTemp. Aunque la regresión lineal trabaja con relaciones lineales, al incluir variables elevadas al cuadrado se le permite representar comportamientos más complejos.

El modelo fue entrenado con validación cruzada de 10 folds y posteriormente evaluado sobre el conjunto de prueba.

Modelo	RMSE	MAE	R ²
LM — FE + Poly	0.4784	0.2205	0.9965

Los resultados muestran que este modelo obtuvo un RMSE de 0.4784, un MAE de 0.2205 y un R² de 0.9965. Esto indica que el modelo explica aproximadamente el 99.65% de la variabilidad de la temperatura media diaria.

Comparado con la regresión lineal base, este modelo mejoró el RMSE y el R^2 , lo que significa que redujo el error general y aumentó ligeramente la capacidad explicativa. Sin embargo, el MAE fue mayor que el de la regresión lineal original, por lo que aunque el modelo mejora en algunas métricas, no representa una mejora absoluta en todos los aspectos.

2.8 - Modelo Mejorado 4: KNN con Ingeniería de Características

Después de aplicar ingeniería de características, se entrenó nuevamente el modelo KNN Regressor con el objetivo de evaluar si las nuevas variables derivadas mejoraban el desempeño predictivo. Este modelo fue seleccionado como una alternativa base no lineal, ya que realiza sus predicciones a partir de la similitud entre observaciones.

KNN funciona buscando los k vecinos más cercanos dentro del conjunto de entrenamiento. Para problemas de regresión, como en este caso, el modelo predice el valor de MeanTemp calculando el promedio de la temperatura media de esos vecinos. Debido a que KNN se basa en distancias, fue necesario trabajar con variables normalizadas, ya que las variables con escalas más grandes podrían dominar el cálculo de cercanía.

En esta etapa se utilizaron las variables generadas mediante ingeniería de características, como TempRange, InteracTemp, LogPrecip, Estacion_num, EsCalorosoMax, MO_sin, MO_cos, DA_sin y DA_cos. Estas variables permiten representar mejor patrones térmicos, temporales y estacionales dentro del dataset.

Durante el entrenamiento se probaron diferentes valores de k , ya que este hiperparámetro determina cuántos vecinos se utilizan para realizar la predicción. Un valor pequeño de k puede hacer que el modelo sea sensible al ruido, mientras que un valor demasiado grande puede suavizar en exceso las predicciones.

Modelo	RMSE	MAE	R^2
Lasso - FE	0.5592	0.1710	0.9952

2.8.1 - KNN con Feature Engineering

Después de aplicar ingeniería de características, se entrenó nuevamente el modelo KNN Regressor, pero ahora utilizando las variables derivadas creadas en la etapa de mejora. El objetivo fue evaluar si estas nuevas variables ayudaban al modelo a encontrar observaciones más similares y mejorar la predicción de la temperatura media diaria MeanTemp.

En esta versión se utilizaron variables como TempRange, InteracTemp, LogPrecip, Estacion_num, EsCalorosoMax, MO_sin, MO_cos, DA_sin y DA_cos, las cuales representan información térmica, temporal y estacional del dataset. Debido a que KNN se

basa en distancias, las variables fueron normalizadas previamente para evitar que alguna característica dominará el cálculo de cercanía por tener una escala mayor.

Durante el entrenamiento se evaluaron diferentes valores de k mediante validación cruzada. El mejor valor encontrado fue:

Parámetro	Valor
Mejor K	9

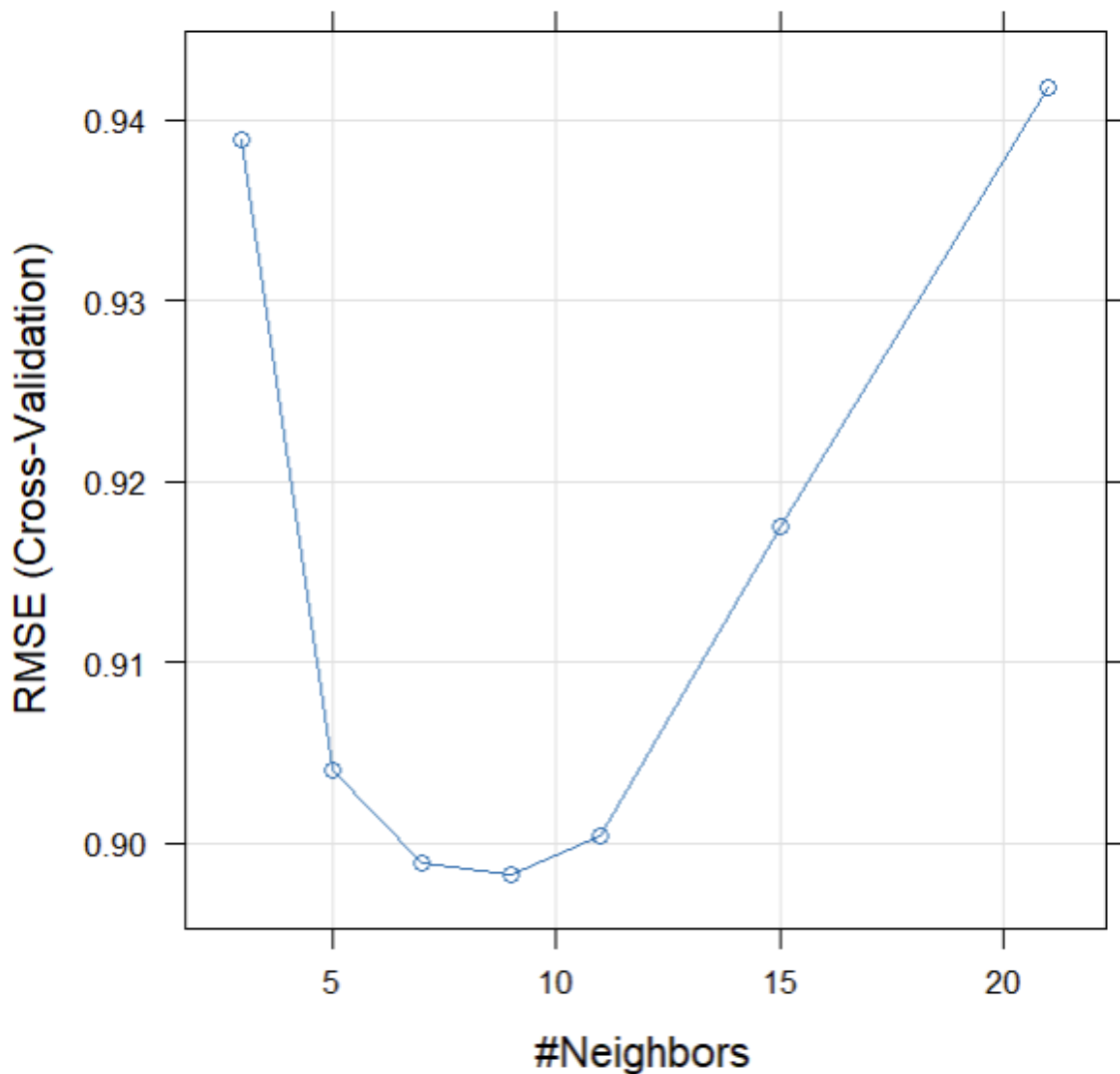
Ahora, los resultados obtenidos del modelo fueron los siguientes:

Modelo	RMSE	MAE	R^2
KNN - FE	0.8734	0.6184	0.9884

El modelo KNN - FE obtuvo un RMSE de 0.8734, un MAE de 0.6184 y un R^2 de 0.9884. Estos resultados indican que el modelo mantiene una capacidad predictiva alta, ya que explica aproximadamente el 98.84% de la variabilidad de MeanTemp.

Sin embargo, al compararlo con el KNN original y el KNN con PCA, el desempeño disminuyó. El RMSE y el MAE aumentaron, mientras que el R^2 bajó. Esto sugiere que, para KNN, agregar más variables mediante ingeniería de características no necesariamente mejoró el modelo. Esto puede deberse a que KNN es sensible a la dimensionalidad: al aumentar el número de variables, el cálculo de distancias puede volverse menos efectivo, especialmente si algunas características nuevas no aportan información clara para identificar vecinos realmente similares.

Aunque la ingeniería de características aportó información útil para otros modelos, en el caso de KNN - FE no produjo una mejora. El mejor valor de $k = 9$ indica que el modelo necesitó considerar más vecinos que en la versión original, posiblemente para compensar el aumento de dimensionalidad y reducir la sensibilidad al ruido.



La gráfica muestra cómo cambia el RMSE de validación cruzada según el número de vecinos utilizados por el modelo KNN. En el eje horizontal se representa el valor de k o número de vecinos, mientras que en el eje vertical se muestra el RMSE, que mide el error del modelo. En este caso, valores más bajos de RMSE indican un mejor desempeño.

Se observa que el error disminuye al pasar de valores muy bajos de k hacia valores intermedios. El mejor resultado se obtiene aproximadamente con $k = 9$, que fue el valor seleccionado por el modelo durante el ajuste de hiperparámetros. Esto significa que, para esta versión con ingeniería de características, el modelo funcionó mejor al considerar nueve vecinos cercanos para calcular la predicción de MeanTemp.

Después de $k = 9$, el RMSE comienza a aumentar nuevamente. Esto indica que usar demasiados vecinos provoca que el modelo suavice en exceso las predicciones, tomando en cuenta registros que ya no son tan similares al dato evaluado. Como resultado, el modelo pierde precisión.

En comparación con el KNN original, el mejor valor de k aumentó de 5 a 9. Esto puede deberse a que la ingeniería de características agregó más variables al dataset, por lo que el modelo necesitó considerar más vecinos para estabilizar sus predicciones. Sin embargo, aunque el ajuste encontró un valor óptimo de k , el desempeño general fue inferior al KNN original y al KNN con PCA, ya que el RMSE final aumentó a 0.8734.

2.8.2 - KNN con Feature Engineering + PCA

Después de entrenar el modelo KNN con las variables derivadas de la ingeniería de características, se entrenó una segunda versión utilizando PCA. El objetivo fue evaluar si la reducción de dimensionalidad ayudaba al modelo KNN a trabajar con una representación más compacta de los datos.

En esta versión, las variables generadas mediante ingeniería de características fueron transformadas en componentes principales. Esto puede ayudar a reducir redundancia entre variables; sin embargo, también existe el riesgo de perder información útil para la predicción de MeanTemp.

Durante el entrenamiento se probaron distintos valores de k , y el mejor valor encontrado fue:

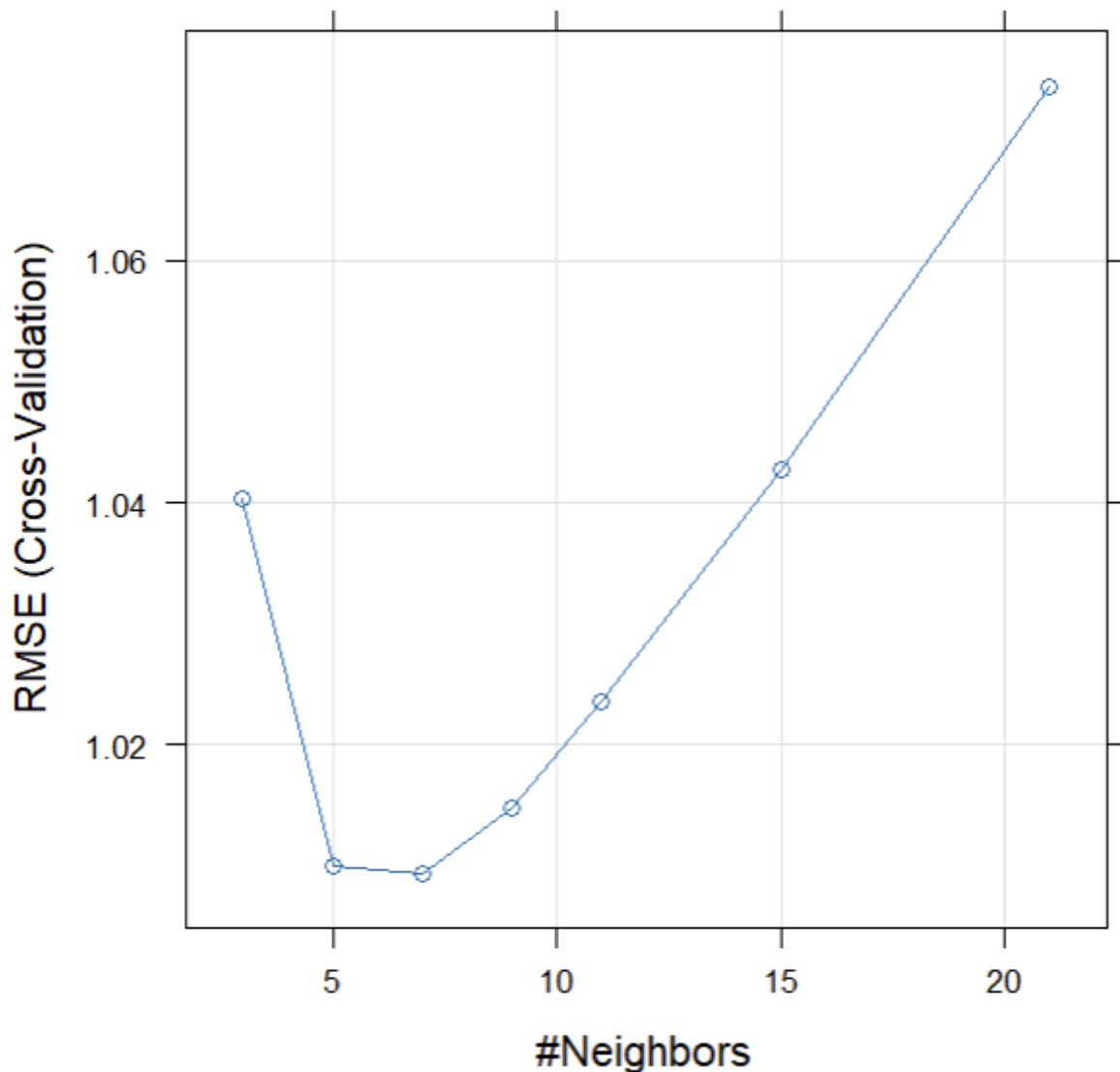
Parámetro	Valor
Mejor K	7

Ahora, los resultados obtenidos del modelo fueron los siguientes:

Modelo	RMSE	MAE	R ²
KNN - FE +PCA	0.9622	0.6552	0.9859

El modelo KNN — FE + PCA obtuvo un RMSE de 0.9622, un MAE de 0.6552 y un R² de 0.9859. Aunque el valor de R² sigue siendo alto, el aumento del RMSE y MAE indica que esta versión tuvo un desempeño inferior en comparación con las otras variantes de KNN.

La gráfica muestra la relación entre el número de vecinos y el RMSE durante la validación cruzada. Se observa que el menor error se alcanza aproximadamente con $k = 7$, por lo que ese fue seleccionado como el mejor valor. Sin embargo, después de ese punto el RMSE aumenta conforme se incrementa el número de vecinos. Esto indica que usar demasiados vecinos suaviza demasiado las predicciones y reduce la precisión del modelo.



En comparación con KNN con Feature Engineering sin PCA, el desempeño disminuyó, ya que el RMSE aumentó de 0.8734 a 0.9622 y el R^2 bajó de 0.9884 a 0.9859.

Esto sugiere que, en este caso, aplicar PCA después de la ingeniería de características no ayudó al modelo KNN. Probablemente, la transformación a componentes principales eliminó o mezcló información útil para encontrar vecinos más similares.

Aunque el PCA puede ser útil para reducir la dimensionalidad, en esta variante no mejoró el desempeño del modelo. Para KNN, la mejor versión sigue siendo la que utiliza las variables originales con PCA, mientras que la combinación de Feature Engineering + PCA presentó el rendimiento más bajo dentro de las versiones evaluadas de KNN.

2.8.3 - Comparación KNN Original, KNN PCA, KNN FE y KNN FE + PCA

Después de entrenar las cuatro versiones del modelo KNN Regressor, se realizó una comparación general para evaluar el efecto de aplicar PCA e ingeniería de características sobre el desempeño del modelo. Las versiones comparadas fueron: KNN con variables originales, KNN con PCA, KNN con Feature Engineering y KNN con Feature Engineering + PCA.

Modelo	Mejor K	RMSE	MAE	R ²
KNN - Original	5	0.7206	0.4573	0.9921
KNN - PCA	5	0.6989	0.3569	0.9926
KNN- FE	9	0.8734	0.6184	0.9884
KNN - FE + PCA	7	0.9622	0.6552	0.9859

Al comparar las cuatro versiones de KNN, se observa que el mejor desempeño fue obtenido por KNN con PCA, con un RMSE de 0.6989, MAE de 0.3569 y R² de 0.9926. Esto indica que, para este modelo, la reducción de dimensionalidad ayudó ligeramente a mejorar las predicciones frente al KNN original.

El KNN original también tuvo un buen rendimiento, con R² de 0.9921, aunque presentó errores mayores. Esto sugiere que PCA permitió trabajar con una representación más compacta y menos redundante de las variables.

En cambio, KNN con Feature Engineering disminuyó su desempeño, obteniendo un RMSE de 0.8734 y MAE de 0.6184. Esto puede deberse a que KNN es sensible al número de variables, ya que al aumentar la dimensionalidad el cálculo de distancias puede volverse menos preciso.

Finalmente, KNN con Feature Engineering + PCA fue la versión con menor rendimiento, con RMSE de 0.9622, MAE de 0.6552 y R² de 0.9859. Esto indica que aplicar PCA después de la ingeniería de características no benefició al modelo.

En conclusión, para KNN la mejor estrategia fue usar PCA sobre las variables originales, mientras que la ingeniería de características no mejoró su desempeño. Esto demuestra que las técnicas de mejora no siempre benefician a todos los modelos por igual.

2.9 - Modelos Avanzados

En esta etapa se incorporaron modelos avanzados con el objetivo de evaluar si algoritmos más complejos pueden mejorar el desempeño obtenido por los modelos base y los modelos con ingeniería de características. A diferencia de los modelos lineales, estos algoritmos tienen la capacidad de capturar relaciones no lineales entre las variables y manejar interacciones de forma automática.

Dentro de esta sección se evaluarán los modelos Random Forest, XGBoost y Gradient Boosting, utilizando las variables generadas después de la ingeniería de características.

2.9.1 - Random Forest

En esta etapa se entrenó un modelo Random Forest utilizando las variables obtenidas después de la ingeniería de características. Este algoritmo se basa en la construcción de múltiples árboles de decisión y combina sus predicciones para obtener un resultado más robusto y preciso. Su principal ventaja es que puede capturar relaciones no lineales e interacciones entre variables sin necesidad de especificarlas manualmente.

Para el entrenamiento se utilizó validación cruzada de 5 folds y se evaluaron distintos valores del hiperparámetro `mtry`, el cual indica cuántas variables se consideran en cada división de los árboles. Además, se configuró el modelo con 200 árboles (`ntree = 200`) y se activó el cálculo de importancia de variables.

Se probaron los siguientes valores de `mtry`:

Hiperparámetro	Valores Evaluados
<code>mtry</code>	2, 4, 6, 8

El mejor valor obtenido fue `mtry = 8`, lo que indica que el modelo tuvo mejor desempeño al considerar 8 variables candidatas en cada división de los árboles.

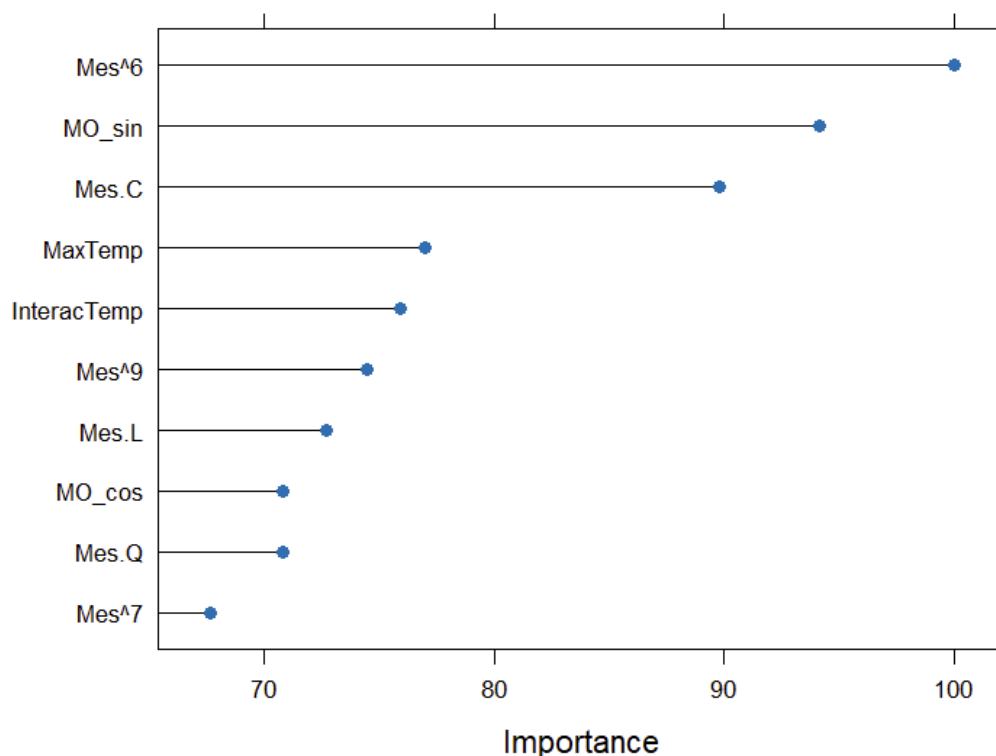
<code>mtry</code>	RMSE	R^2	MAE
2	1.9237	0.9744	1.1639
4	0.4686	0.9970	0.2029
6	0.3549	0.9981	0.1019
8	0.3379	0.9982	0.0859

Los resultados muestran que el mejor desempeño se obtuvo con $mtry = 8$, ya que presentó el menor RMSE (0.3379) y el menor MAE (0.0859), además de un valor alto de R^2 (0.9982) durante la validación cruzada. Esto indica que el modelo tuvo un mejor comportamiento al considerar 8 variables candidatas en cada división de los árboles.

Modelo	mtry	RMSE	MAE	R^2
Random Forest - FE	8	0.3832	0.0810	0.9978

Al evaluar el modelo final sobre el conjunto de prueba, Random Forest obtuvo un RMSE de 0.3832, un MAE de 0.0810 y un R^2 de 0.9978. Estos resultados indican un desempeño muy alto, ya que el modelo logra explicar aproximadamente el 99.78% de la variabilidad de MeanTemp y presenta un error absoluto promedio muy bajo.

Random Forest — Importancia de Variables



La gráfica muestra las variables que tuvieron mayor influencia en las predicciones del modelo Random Forest. En este tipo de modelo, la importancia de variables permite identificar qué características fueron más utilizadas por los árboles para reducir el error de predicción y mejorar la estimación de la variable objetivo MeanTemp.

Se observa que algunas variables relacionadas con el mes y la estacionalidad, como Mes^6, MO_sin y Mes.C, aparecen entre las más importantes. Esto indica que el comportamiento temporal tiene un papel relevante en la predicción de la temperatura media diaria, lo cual coincide con el análisis temporal realizado previamente, donde se observó un patrón estacional claro.

También destacan variables térmicas como MaxTemp e InteracTemp. La presencia de MaxTemp confirma que la temperatura máxima diaria es una de las variables más relacionadas con MeanTemp. Por otro lado, InteracTemp, creada durante la ingeniería de características como una interacción entre MaxTemp y MinTemp, demuestra que las variables derivadas aportaron información útil al modelo.

En conclusión, Random Forest fue uno de los modelos con mejor desempeño dentro del proyecto. Su capacidad para capturar relaciones no lineales e interacciones entre variables permitió obtener errores bajos y un R^2 alto, superando a varios modelos base y mejorados evaluados previamente.

2.9.2 - XGBoost

En esta etapa se entrenó un modelo XGBoost para regresión utilizando las variables generadas después de la ingeniería de características. Debido a problemas de compatibilidad al entrenarlo mediante caret, el modelo se implementó directamente con la función `xgb.train()`, utilizando matrices numéricas y el objetivo `reg:squarederror`.

Durante el entrenamiento se configuraron 200 iteraciones y se utilizaron parámetros como profundidad máxima, tasa de aprendizaje, subsampling y selección parcial de columnas. A lo largo de las iteraciones, el error RMSE disminuyó de forma progresiva tanto en entrenamiento como en prueba, pasando de valores superiores a 7 al inicio hasta aproximadamente 0.3900 en el conjunto de prueba al finalizar el entrenamiento. Esto indica que el modelo fue aprendiendo patrones relevantes conforme avanzan las iteraciones.

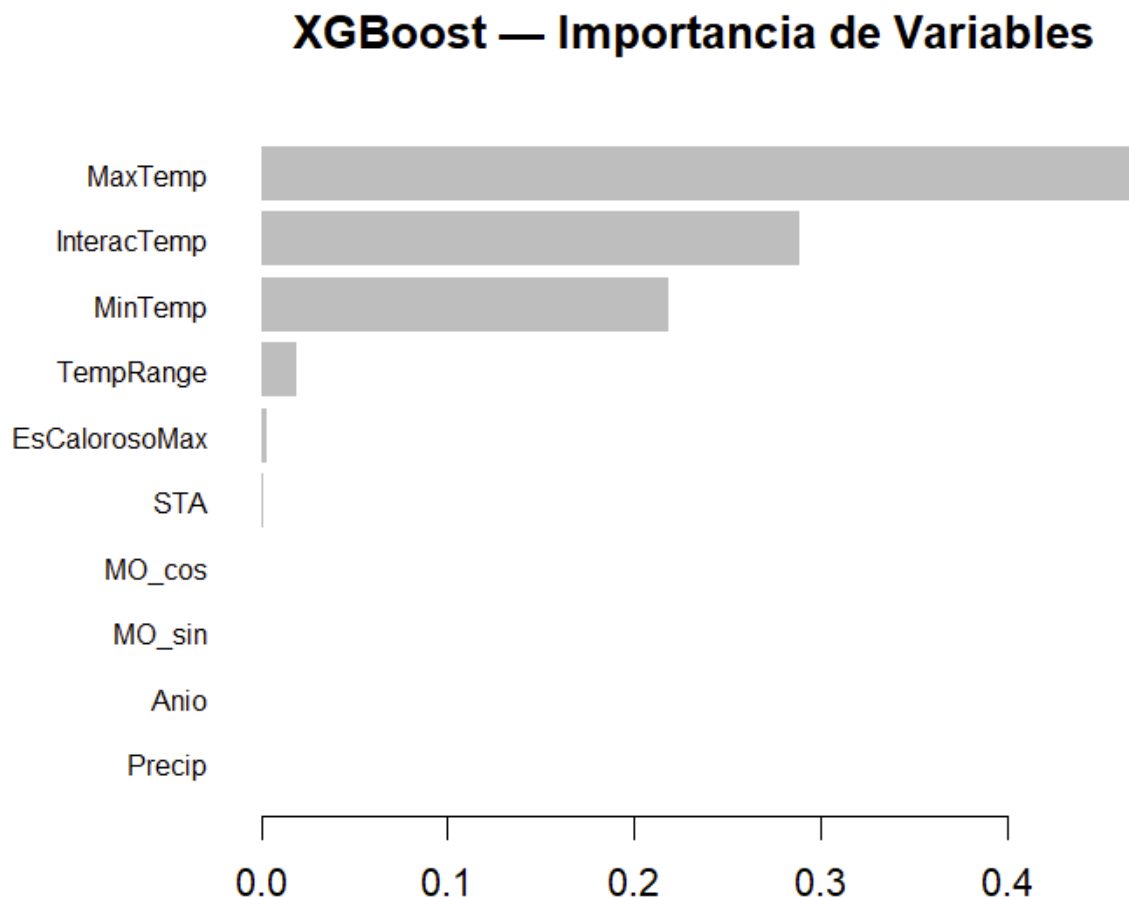
Los resultados obtenidos fueron los siguientes:

Modelo	RMSE	MAE	R^2
XGBoost -FE	0.3900	0.1166	0.9977

El modelo obtuvo un RMSE de 0.3900, un MAE de 0.1166 y un R^2 de 0.9977. Estos resultados indican un desempeño muy alto, ya que el modelo explica aproximadamente el 99.77% de la variabilidad de la temperatura media diaria. Además, sus errores son bajos, lo que muestra que sus predicciones son cercanas a los valores reales de MeanTemp.

La gráfica de importancia de variables muestra cuáles características tuvieron mayor peso en las predicciones del modelo XGBoost. Se observa que las variables más importantes fueron MaxTemp, InteracTemp y MinTemp. Esto confirma que las variables relacionadas directamente con la temperatura son las más relevantes para estimar MeanTemp.

También destaca InteracTemp, una variable creada durante la ingeniería de características mediante la interacción entre MaxTemp y MinTemp. Su alta importancia indica que la ingeniería de características sí aportó información útil al modelo. Otras variables como TempRange, EsCalorosoMax, STA, MO_cos y MO_sin tuvieron menor influencia, pero aportan información complementaria sobre variabilidad térmica, estación meteorológica y patrones temporales.



La gráfica de importancia de variables muestra que XGBoost basó principalmente sus predicciones en variables térmicas como MaxTemp, InteracTemp y MinTemp. Esto indica que la temperatura máxima, la temperatura mínima y la interacción entre ambas son los factores más relevantes para estimar MeanTemp. La presencia de InteracTemp entre las variables más importantes confirma que la ingeniería de características contribuyó positivamente al modelo.

En conclusión, XGBoost con Feature Engineering presentó un desempeño sobresaliente, con un R^2 de 0.9977 y errores bajos. Su rendimiento fue muy cercano al de Random Forest, aunque con un RMSE ligeramente menor en tus resultados actuales. Esto lo convierte en uno de los modelos más competitivos dentro de la etapa de modelos avanzados.

2.9.3 - Gradient Boosting (GBM)

En esta etapa se entrenó un modelo Gradient Boosting Machine (GBM) utilizando las variables generadas después de la ingeniería de características. Este modelo construye árboles de decisión de manera secuencial, donde cada nuevo árbol intenta corregir los errores cometidos por los árboles anteriores. Por esta razón, GBM puede capturar relaciones no lineales e interacciones entre variables.

Para el entrenamiento se evaluaron diferentes combinaciones de hiperparámetros mediante validación cruzada de 5 folds. Los parámetros ajustados fueron el número de árboles, la profundidad máxima de los árboles, la tasa de aprendizaje y el número mínimo de observaciones por nodo.

Los resultados obtenidos fueron los siguientes:

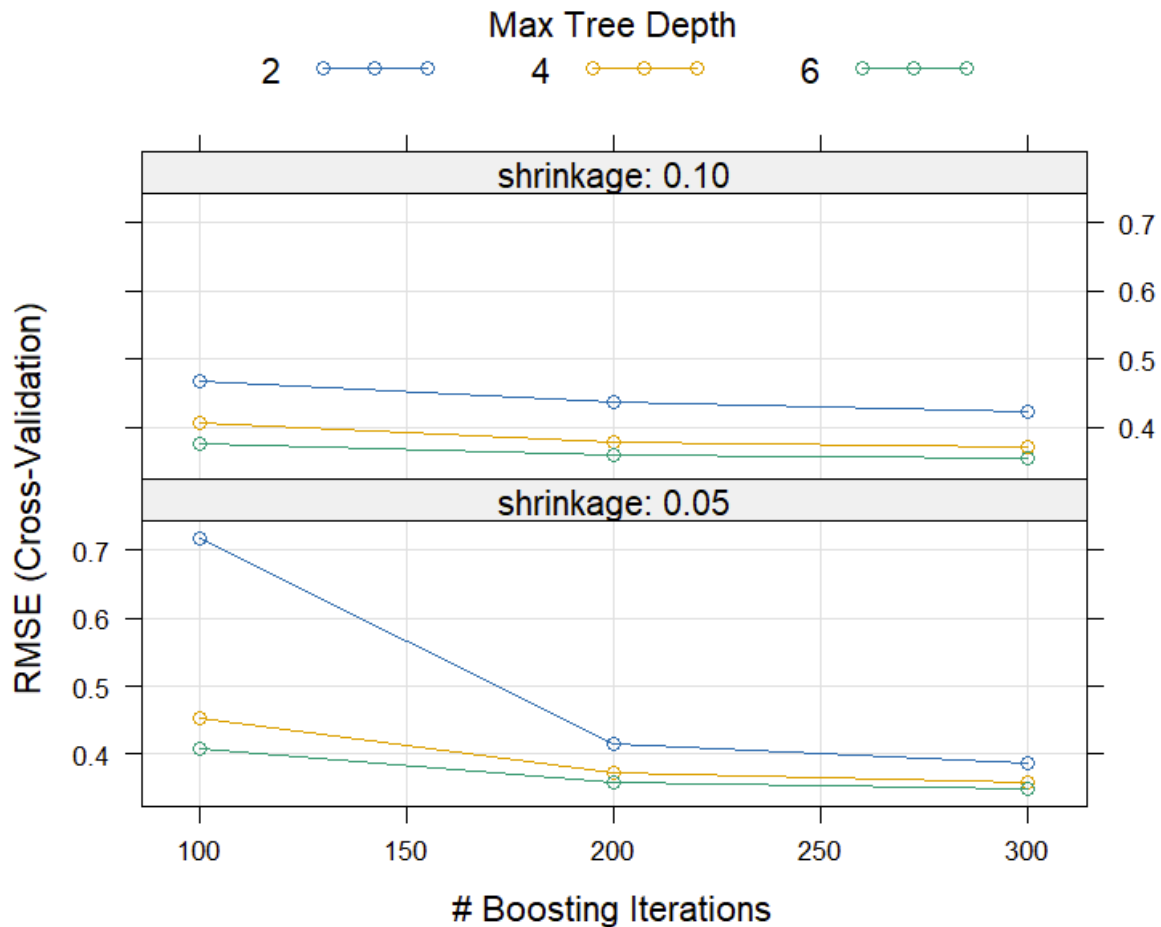
Parámetro	Valor
n.trees	300
interaction.depth	6
shrinkage	0.05
n.minobsinnode	10

La gráfica de ajuste muestra cómo cambia el RMSE según el número de iteraciones, la profundidad de los árboles y el valor de shrinkage. Se observa que el mejor desempeño se obtiene con 300 árboles, profundidad 6 y una tasa de aprendizaje de 0.05, ya que esta combinación logra el menor error durante la validación cruzada.

Resultados del modelo:

Modelo	RMSE	MAE	R ²
GBM - FE	0.4090	0.1642	0.9974

El modelo obtuvo un RMSE de 0.4090, un MAE de 0.1642 y un R² de 0.9974. Estos resultados indican que el modelo tiene un desempeño muy alto, ya que explica aproximadamente el 99.74% de la variabilidad de la temperatura media diaria. Además, sus valores de error son bajos, lo que significa que las predicciones se acercan bastante a los valores reales de MeanTemp.



La gráfica muestra el comportamiento del error RMSE durante la validación cruzada para distintas configuraciones del modelo GBM. En el eje horizontal se representa el número de árboles o iteraciones de boosting, mientras que en el eje vertical se muestra el error RMSE. Las líneas representan diferentes profundidades máximas de los árboles y los paneles separan los valores de shrinkage.

Se observa que, en general, al aumentar el número de árboles el RMSE disminuye, lo que indica que el modelo mejora conforme incorpora más iteraciones. También se aprecia que una profundidad mayor, especialmente `interaction.depth = 6`, produce mejores resultados. Finalmente, el valor de `shrinkage = 0.05` obtuvo el mejor desempeño, mostrando que una tasa de aprendizaje más baja permitió al modelo aprender de forma más gradual y precisa.

En conclusión, el modelo GBM con Feature Engineering presentó un rendimiento sobresaliente, con un R^2 de 0.9974 y errores bajos. Aunque no superó completamente a XGBoost ni a Random Forest en todas las métricas, se mantiene como uno de los modelos avanzados con mejor desempeño dentro del proyecto. Su capacidad para combinar múltiples árboles de forma secuencial le permitió capturar patrones complejos en los datos meteorológicos.

3.0 - Ensambles

3.0.1 - Ensemble Random Forest + XGBoost

En esta etapa se construyó un ensemble combinando las predicciones de los modelos Random Forest y XGBoost. El objetivo fue evaluar si al promediar las predicciones de dos modelos avanzados se podía obtener un resultado más estable y reducir el error de predicción.

El ensemble se realizó mediante un promedio simple entre las predicciones de ambos modelos y resultados obtenidos fueron:

Modelo	RMSE	MAE	R ²
Ensamble RF + XGBoost	0.3952	0.1094	0.9976

El ensemble obtuvo un RMSE de 0.3952, un MAE de 0.1094 y un R² de 0.9976. Estos resultados indican un desempeño muy alto, ya que el modelo combinado explica aproximadamente el 99.76% de la variabilidad de la temperatura media diaria.

Al compararlo con los modelos individuales, el ensemble logró un MAE menor que XGBoost, lo que significa que redujo el error absoluto promedio. Sin embargo, su RMSE fue ligeramente mayor que el de XGBoost individual, por lo que el modelo XGBoost sigue siendo el mejor si se toma como criterio principal el RMSE.

El ensemble Random Forest + XGBoost mostró un rendimiento competitivo y estable. Aunque no superó completamente a XGBoost en todas las métricas, sí mejoró el MAE, lo que indica que combinar modelos puede ayudar a reducir ciertos tipos de error y generar predicciones más equilibradas.

3.0.2 - Ensemble XGBoost + GBM

Ahora se construyó un ensemble combinando las predicciones de los modelos XGBoost y GBM. Ambos modelos pertenecen a la familia de algoritmos de *boosting*, por lo que construyen árboles de decisión de forma secuencial, tratando de corregir los errores de los modelos anteriores.

El objetivo de este ensemble fue evaluar si la combinación de dos modelos avanzados basados en boosting podía generar predicciones más estables y reducir el error en comparación con los modelos individuales.

El ensemble se realizó mediante un promedio simple entre las predicciones de ambos modelos y los resultados obtenidos fueron:

Modelo	RMSE	MAE	R ²
Ensamble XGBoost + GBM	0.3925	0.1343	0.9977

El ensamble obtuvo un RMSE de 0.3925, un MAE de 0.1343 y un R^2 de 0.9977. Estos resultados indican un desempeño muy alto, ya que el modelo combinado logra explicar aproximadamente el 99.77% de la variabilidad de la temperatura media diaria.

Al compararlo con XGBoost individual, el ensamble mantiene prácticamente el mismo valor de R^2 , pero su RMSE y MAE son ligeramente mayores. Esto indica que, aunque el ensamble es competitivo y estable, XGBoost por sí solo sigue teniendo un desempeño ligeramente superior en este caso.

El ensamble XGBoost + GBM presentó un rendimiento muy bueno, pero no logró superar completamente al modelo XGBoost individual. Aun así, demuestra que combinar modelos de boosting puede generar resultados sólidos y cercanos al mejor modelo obtenido.

3.0.3 - Random Forest + XGBoost + GBM

En esta etapa se construyó un ensamble ponderado combinando las predicciones de los modelos Random Forest, XGBoost y GBM. A diferencia de un promedio simple, en este caso se asignaron pesos diferentes a cada modelo según su desempeño individual.

Debido a que XGBoost fue el modelo individual con mejor rendimiento, se le asignó un mayor peso dentro del ensamble. La combinación utilizada fue: $0.25(\text{Random Forest}) + 0.50(\text{XGBoost}) + 0.25(\text{GBM})$.

Los resultados obtenidos fueron:

Modelo	RMSE	MAE	R^2
Ensamble RF + XGBoost + GBM	0.3893	0.1178	0.9977

El ensamble ponderado obtuvo un RMSE de 0.3893, un MAE de 0.1178 y un R^2 de 0.9977. Estos resultados indican un desempeño muy alto, ya que el modelo combinado explica aproximadamente el 99.77% de la variabilidad de la temperatura media diaria.

Además, este ensamble logró un RMSE ligeramente menor que XGBoost individual, que tenía un RMSE de 0.3900. Esto significa que la combinación ponderada de modelos permitió reducir ligeramente el error general de predicción. Sin embargo, el MAE fue un poco mayor que el de XGBoost individual, por lo que la mejora principal se observa en el RMSE.

El ensamble ponderado Random Forest + XGBoost + GBM fue el mejor resultado obtenido en términos de RMSE. Esto demuestra que combinar modelos avanzados puede mejorar ligeramente el desempeño predictivo, especialmente cuando se asigna mayor peso al modelo con mejor rendimiento individual.

3.1 - Comparación Final de Todos los Modelos

Por último, se realizó la comparación final de todos los modelos entrenados durante el proyecto, utilizando las métricas RMSE, MAE y R^2 . El objetivo fue identificar qué modelo presentó el mejor desempeño para predecir la variable objetivo MeanTemp.

El RMSE y el MAE permiten medir el error de predicción, por lo que valores más bajos indican un mejor modelo. Por otro lado, el R^2 mide qué proporción de la variabilidad de la temperatura media diaria es explicada por el modelo, por lo que valores más cercanos a 1 representan un mejor ajuste.

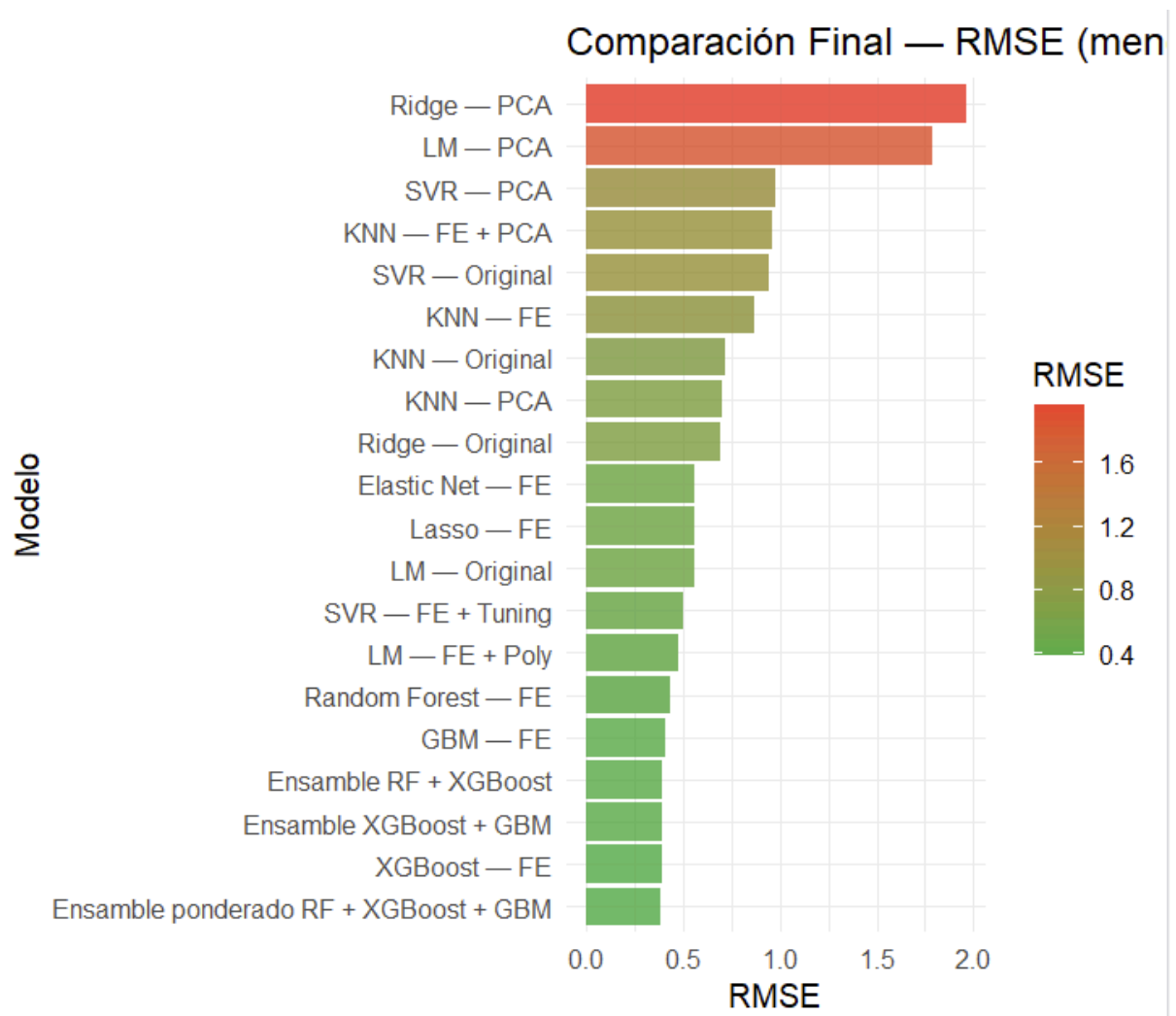
Modelo	RMSE	MAE	R^2
XGBoost - FE	0.3900	0.1166	0.9977
GBM - FE	0.4090	0.1642	0.9974
Random Forest - FE	0.4342	0.1242	0.9971
LM - FE + Poly	0.4784	0.2206	0.9965
SVR - FE + Tuning	0.5021	0.2899	0.9962
LM - Original	0.5572	0.1676	0.9953
Lasso - FE	0.5592	0.1710	0.9952
Elastic Net - FE	0.5593	0.1768	0.9952
Ridge - Original	0.6922	0.3442	0.9927
SVR - Original	0.9479	0.3781	0.9863
SVR - PCA	0.9804	0.3859	0.9853
LM - PCA	1.7890	0.9103	0.9512
Ridge - PCA	1.9620	1.1967	0.9413
RF + XGBoost + GBM	0.3893	0.1178	0.9977
XGBoost + GBM	0.3925	0.1343	0.9977
RF + XGBoost	0.3952	0.1094	0.9976
KNN - PCA	0.6989	0.3569	0.9926
KNN - Original	0.7206	0.4573	0.9921
KNN -FE	0.8734	0.6184	0.9884
KNN - FE + PCA	0.9622	0.6552	0.9859

3.1.1 - RMSE

La gráfica de RMSE muestra que el modelo con menor error fue XGBoost - FE, con un valor de 0.3900. Esto significa que, en promedio, sus predicciones se desviaron poco respecto a la temperatura real.

Después de XGBoost, los modelos con mejor desempeño fueron GBM - FE y Random Forest - FE, ambos modelos avanzados basados en árboles. Esto indica que los algoritmos capaces de capturar relaciones no lineales e interacciones entre variables lograron mejores resultados que los modelos base.

En contraste, las versiones con PCA, especialmente Ridge - PCA y LM - PCA, presentaron los valores de RMSE más altos. Esto sugiere que, en este caso, la reducción de dimensionalidad provocó pérdida de información relevante para la predicción de MeanTemp.

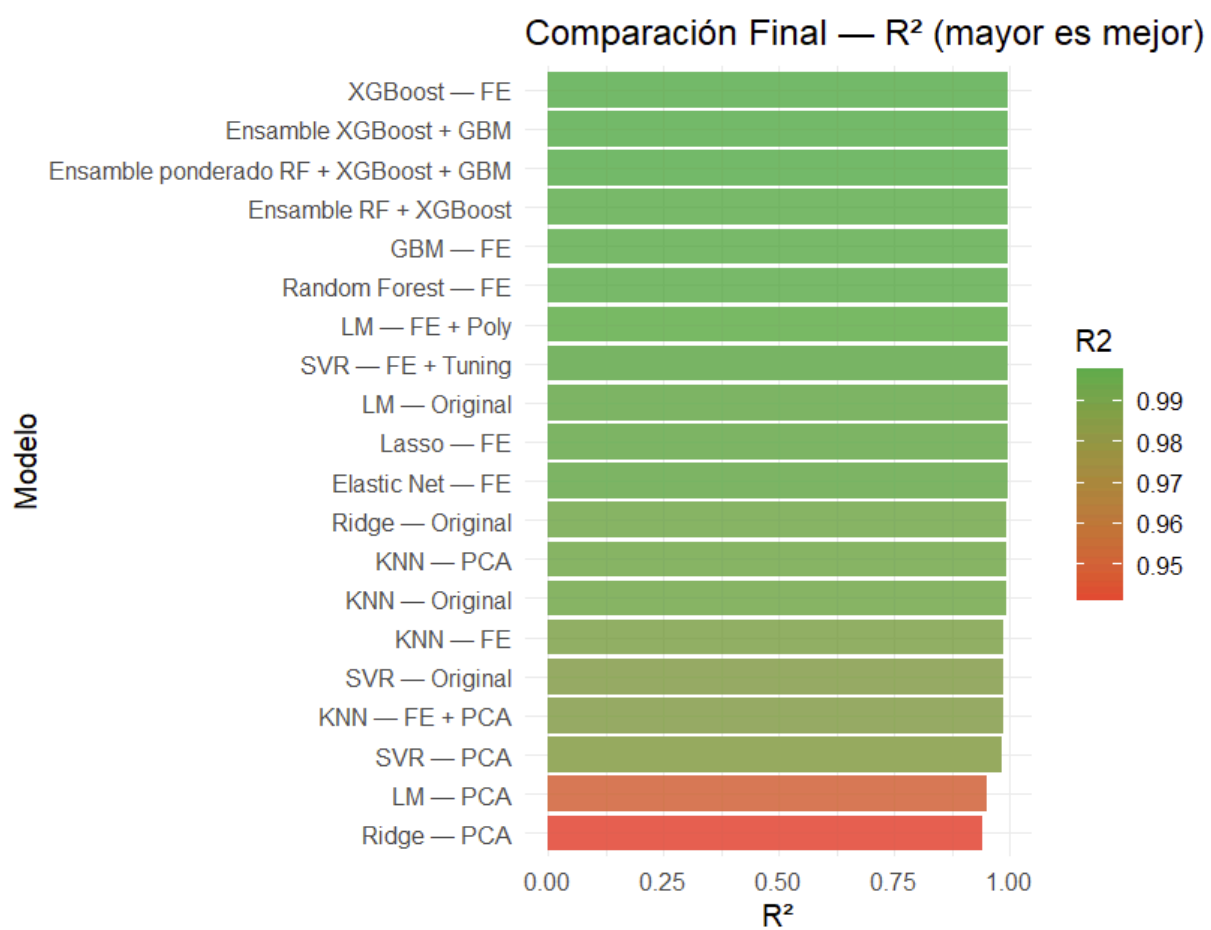


3.1.2 - Gráfica R²

La gráfica de R² confirma lo observado en el RMSE. Los modelos avanzados XGBoost, GBM y Random Forest obtuvieron valores superiores a 0.997, lo que indica que explican más del 99.7% de la variabilidad de la temperatura media diaria.

El modelo XGBoost - FE obtuvo el mejor resultado general con un R² de 0.9977, seguido por GBM - FE con 0.9974 y Random Forest - FE con 0.9971. Esto demuestra que los modelos avanzados aprovecharon de mejor manera las variables generadas mediante ingeniería de características.

Por otro lado, los modelos con PCA tuvieron un desempeño inferior, especialmente Ridge - PCA y LM - PCA, con valores de R² más bajos. Esto refuerza la conclusión de que PCA no aportó una mejora significativa para este problema.

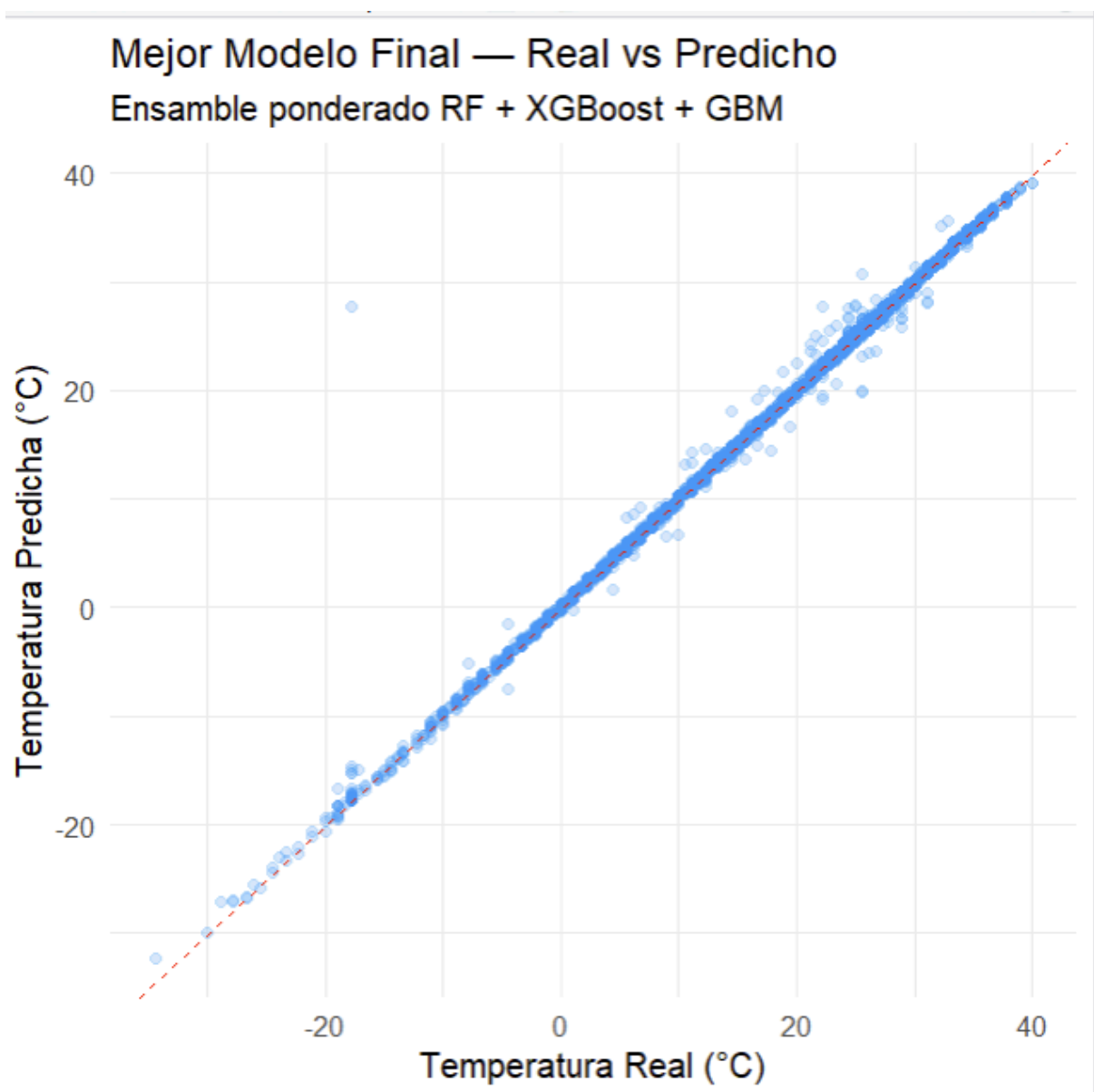


3.1.3 - Gráfica Real vs Predicho

La gráfica Real vs Predicho corresponde al mejor modelo, XGBoost - FE. En ella se comparan las temperaturas reales contra las temperaturas predichas por el modelo.

Se observa que la mayoría de los puntos se encuentran muy cerca de la línea roja diagonal, la cual representa una predicción ideal. Esto significa que las predicciones del modelo son muy similares a los valores reales. Aunque existen algunos puntos alejados, la concentración general alrededor de la línea indica un ajuste muy bueno.

Esta visualización confirma que XGBoost - FE logró predecir correctamente la temperatura media diaria en la mayoría de los casos.



3.2 - Conclusiones

A partir del desarrollo de este proyecto se logró construir y evaluar un flujo completo de Machine Learning orientado a la predicción de la temperatura media diaria, utilizando como variable objetivo MeanTemp. El proceso inició con una etapa de análisis exploratorio de datos, donde se revisó la estructura general del dataset, la calidad de la información, la presencia de valores faltantes, la distribución de la variable objetivo y la relación entre las variables disponibles. Esta etapa fue fundamental para comprender el comportamiento de los datos antes de aplicar cualquier modelo de regresión.

Durante el análisis inicial se identificó que algunas variables presentaban un porcentaje muy alto de valores faltantes, incluso varias de ellas con el 100% de datos nulos. Por esta razón, fue necesario realizar una limpieza del dataset y conservar únicamente aquellas variables con información suficiente y relevante para el modelado. También se detectó que variables como MaxTemp y MinTemp tenían una relación muy fuerte con MeanTemp, lo cual permitió establecer desde el inicio que las variables térmicas serían fundamentales para la predicción.

En la primera etapa de modelado se entrenaron modelos base como Regresión Lineal Múltiple, SVR y Ridge Regression, tanto con variables originales como con PCA. Estos modelos permitieron establecer una línea base para comparar el desempeño de técnicas más avanzadas. Los resultados mostraron que la Regresión Lineal original obtuvo un rendimiento alto, con errores bajos y un valor de R^2 elevado. Sin embargo, las versiones con PCA no siempre mejoran el desempeño, lo que indica que la reducción de dimensionalidad no fue completamente beneficiosa para este problema, ya que pudo eliminar parte de la información relevante contenida en las variables originales.

Posteriormente, en la segunda parte del proyecto, se aplicó ingeniería de características con el objetivo de enriquecer el dataset y mejorar la capacidad predictiva de los modelos. Se generaron variables como TempRange, InteracTemp, LogPrecip, Estacion_num, y variables cíclicas como MO_sin, MO_cos, DA_sin y DA_cos. Estas nuevas características permiten representar de mejor manera aspectos como la variación térmica diaria, la interacción entre temperaturas, la estacionalidad y los patrones temporales. Los resultados obtenidos posteriormente confirmaron que la ingeniería de características aportó información útil, especialmente en los modelos avanzados.

En cuanto a los modelos mejorados, se evaluaron técnicas como Lasso, Elastic Net, SVR con ajuste de hiperparámetros y Regresión Lineal con términos polinomiales. Estos modelos lograron mantener un desempeño alto, aunque no todos superaron a los modelos base. Por ejemplo, Lasso y Elastic Net obtuvieron resultados muy similares a la regresión lineal original, mientras que la regresión lineal con términos polinomiales logró mejorar el RMSE y el R^2 . Por su parte, el SVR con tuning también mostró una mejora importante respecto al SVR base, demostrando que el ajuste de hiperparámetros puede incrementar considerablemente el rendimiento de un modelo.

Finalmente, se implementaron modelos avanzados basados en árboles, como Random Forest, XGBoost y GBM. Estos modelos fueron los que obtuvieron los mejores resultados generales, ya que tienen la capacidad de capturar relaciones no lineales e interacciones

complejas entre variables. Dentro de esta etapa, el modelo con mejor desempeño fue XGBoost con Feature Engineering, con un RMSE de 0.3900, un MAE de 0.1166 y un R^2 de 0.9977. Estos valores indican que el modelo fue capaz de explicar aproximadamente el 99.77% de la variabilidad de la temperatura media diaria, además de mantener un error de predicción muy bajo.

La gráfica de comparación final confirmó que los modelos avanzados superaron a los modelos base y a las versiones con PCA. En particular, XGBoost, GBM y Random Forest se ubicaron como los modelos con mejor desempeño, mientras que las versiones con PCA, como Ridge - PCA y LM - PCA, presentaron los errores más altos. Esto permite concluir que, para este dataset, fue más conveniente conservar las variables originales y enriquecidas mediante ingeniería de características, en lugar de reducirlas mediante componentes principales.

Además, la gráfica de Real vs Predicho del modelo XGBoost mostró que la mayoría de los puntos se ubicaron muy cerca de la línea diagonal ideal. Esto significa que las predicciones del modelo fueron muy similares a los valores reales de temperatura, confirmando visualmente el buen desempeño obtenido en las métricas cuantitativas.

En conclusión, el proyecto permitió comprobar que una adecuada limpieza de datos, acompañada de ingeniería de características y modelos avanzados, puede mejorar significativamente el desempeño predictivo. El modelo XGBoost — FE se considera la mejor opción final para la predicción de MeanTemp, ya que presentó el menor error y la mayor capacidad explicativa entre todos los modelos evaluados. Esto demuestra que los algoritmos avanzados basados en árboles son especialmente adecuados para este tipo de problema, donde existen relaciones no lineales, patrones temporales y múltiples variables climáticas que influyen en la temperatura media diaria.